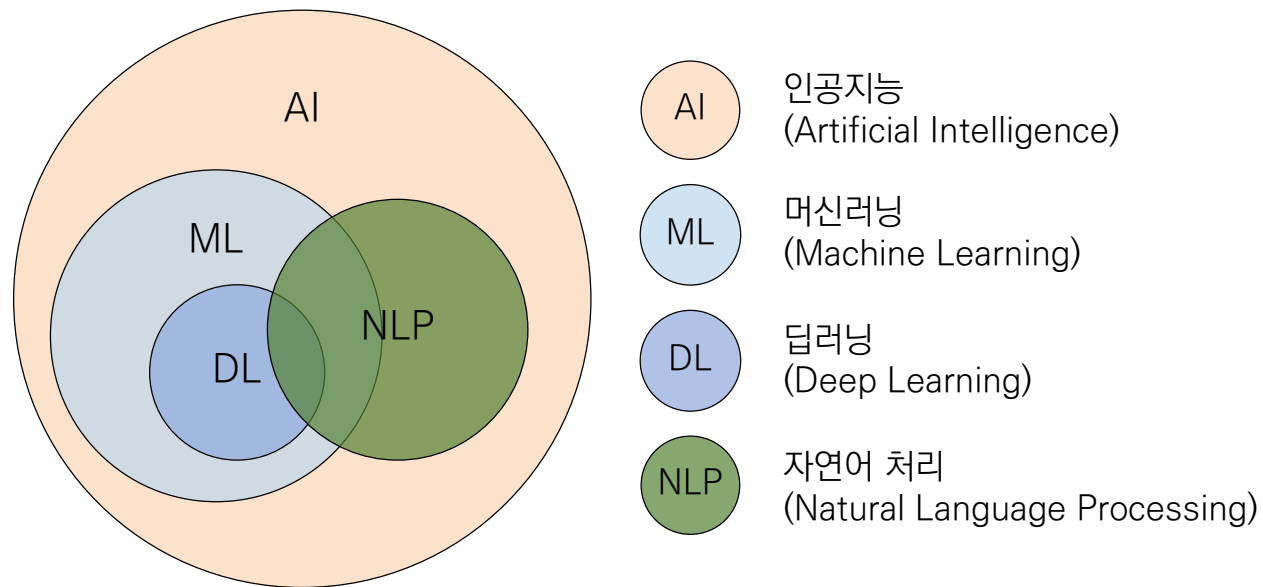


- #1 NLTK 자연어 처리 패키지
- #2 한글 형태소 분석과 워드 클라우드
- #3 연관분석
- #4 케라스를 이용한 인공지능망 구현
- #5 RNN

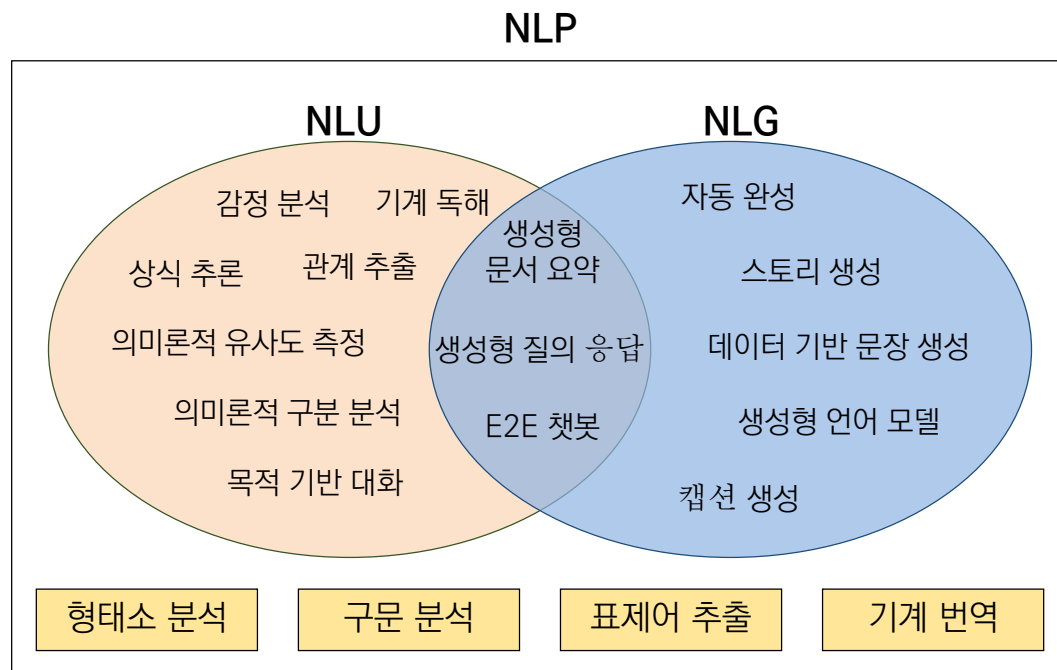


자연어 처리 영역



자연어와 자연어처리 분야

- 자연어는 인간이 상대방과 의사소통을 위해서 사용하는 언어(한국어, 영어, 중국어 등)
- 자연어처리(NLP; Natural Language Processing)는 사람의 자연어를 컴퓨터가 이해하고 처리할 수 있도록 하는 인공지능의 한 분야



1. NLTK 자연어 처리 패키지

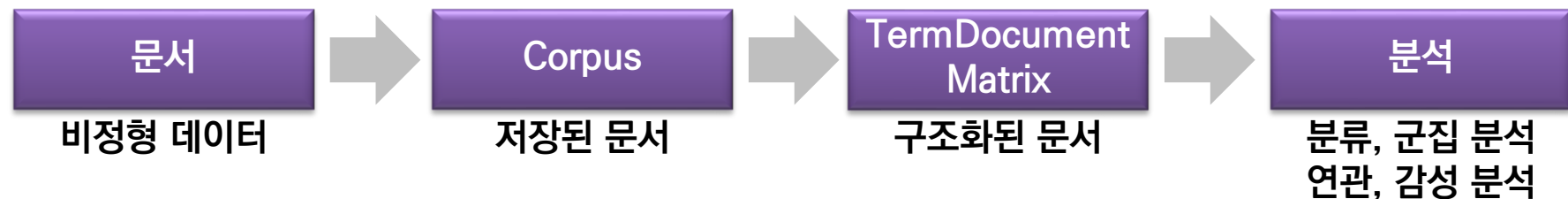


텍스트 마이닝 개요

1. NLTK 자연어 처리 패키지

텍스트 마이닝(Text Mining)은 데이터 마이닝(Data Mining)의 한 분야에 속하며 뉴스 기사, SNS 글 등 **자연어에서 의미 있는 정보를 찾는 것**

텍스트 마이닝은 다양한 형태의 **비정형 문서 데이터로부터 문서별 단어의 행렬(Matrix)**을 만든 후, 여러 가지 분석 기법과 데이터 마이닝 기법을 사용하여 **통찰(Insight)**을 얻거나 **의사결정을 지원**하기 위해 사용



자연어 처리와 텍스트 마이닝

1. NLTK 자연어 처리 패키지

자연어 처리 학습 주제

- 텍스트 전처리 : 토큰화, 정제, 형태소 분석, 불용어 처리, 레이블 인코딩
- 개수 기반 단어 표현 : 문장 내에서 단어들의 빈도수를 측정해서 이를 기반으로 데이터를 분석할 수 있는 형태로 만드는 것
- 문서 유사도(Document Similarity) : 단어들을 수치화 한 후 이를 기반으로 단어들 사이의 거리를 계산해서 문서 간의 단어들의 차이를 계산하는 것
- 토픽 모델링(Topic Modeling) : 텍스트 본문의 숨겨진 의미 구조를 발견하기 위해 사용되는 텍스트 마이닝 기법
- 연관 분석(Association) : 문서 내의 단어들을 이용해서 연관 분석을 실시
- 딥러닝을 이용한 자연어 처리 : RNN의 인공신경망 알고리즘을 이용해서 딥러닝으로 자연어 처리
- 워드 임베딩(Word Imbedding) : Word2vec 패키지를 이용해서 단어를 벡터로 표현하는 방법으로 희소 표현에서 밀집 표현으로 변환하는 것
- 텍스트 분류(Text Classification) : 텍스트를 입력으로 받아, 텍스트가 어떤 종류의 범주(Class)에 속하는지를 구분하는 작업
- 태깅(Tagging) : 각 단어가 어떤 유형에 속해 있는지를 알아내는 것

NLTK(Natural Language Toolkit)

1. NLTK 자연어 처리 패키지

교육용으로 개발된 자연어 처리와 문서 분석용 파이썬 패키지

NLTK 패키지가 제공하는 주요 기능

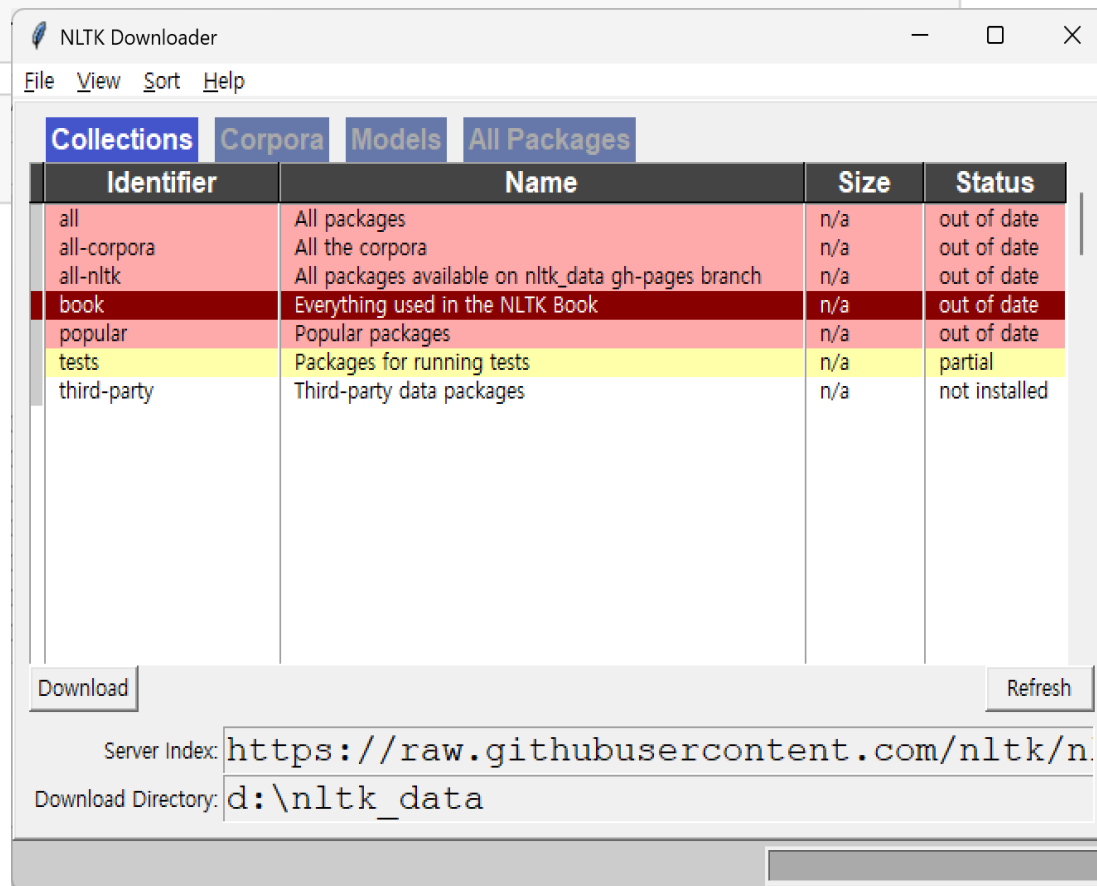
- 말뭉치(corpus)
- 토큰 생성(tokenizing)
- 형태소 분석(morphological analysis)
- 품사 태깅(POS tagging)

말뭉치(Corpus)

1. NLTK 자연어 처리 패키지

```
1 import nltk
```

```
1 nltk.download()
```



Penn Treebank 데이터

1. NLTK 자연어 처리 패키지

펜 트리뱅크 : NLTK를 설명할 목적으로 사용하며 토큰화, 태깅, 청킹, 파싱에 사용

400만 어절 규모의 펜 트리뱅크(Penn Treebankk, 1990-92, 미국)

The screenshot shows the NLTK Downloader application window. It has a menu bar with 'File', 'View', 'Sort', and 'Help'. Below the menu bar are four tabs: 'Collections', 'Corpora', 'Models', and 'All Packages'. The 'Corpora' tab is selected, displaying a table of available corpora. The table has four columns: 'Identifier', 'Name', 'Size', and 'Status'. The 'treebank' package is highlighted in green and marked as 'installed'. Below the table are buttons for 'Download' and 'Refresh'. At the bottom, there are fields for 'Server Index' and 'Download Directory', and a status bar indicating 'Package treebank is up-to-date!'.

Identifier	Name	Size	Status
state_union	C-Span State of the Union Address Corpus	789.8 KB	installed
stopwords	Stopwords Corpus	33.5 KB	installed
subjectivity	Subjectivity Dataset v1.0	509.4 KB	not installed
swadesh	Swadesh Wordlists	22.3 KB	installed
switchboard	Switchboard Corpus Sample	772.6 KB	not installed
timit	TIMIT Corpus Sample	21.2 MB	installed
toolbox	Toolbox Sample Files	244.7 KB	installed
treebank	Penn Treebank Sample	1.7 MB	installed
twitter_samples	Twitter Samples	15.3 MB	not installed
udhr	Universal Declaration of Human Rights Corpus	1.1 MB	installed
udhr2	Universal Declaration of Human Rights Corpus (Unicode)	1.6 MB	not installed
unicode_samples	Unicode Samples	1.2 KB	installed
universal_treebanks_v20	Universal Treebanks Version 2.0	24.7 MB	not installed
verbnet	VerbNet Lexicon, Version 2.1	316.1 KB	not installed
verbnet3	VerbNet Lexicon, Version 3.3	470.7 KB	not installed
webtext	Web Text Corpus	631.1 KB	installed

Server Index: https://raw.githubusercontent.com/nltk/nltk_data/gh-pages/index.xml

Download Directory: d:\nltk_data

Package treebank is up-to-date!

book 데이터

1. NLTK 자연어 처리 패키지

```
1 nltk.download("book", quiet=True)
```

True

```
1 from nltk.book import *
```

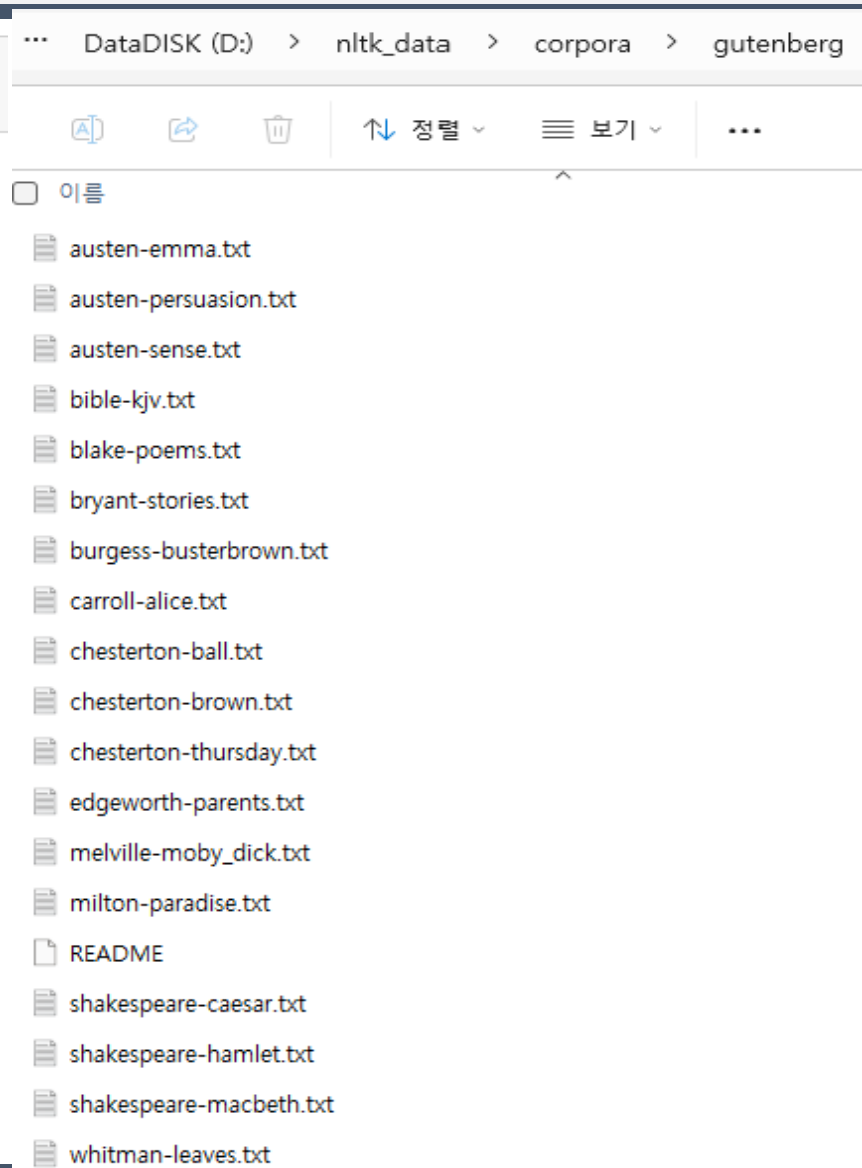
```
*** Introductory Examples for the NLTK Book ***
Loading text1, ..., text9 and sent1, ..., sent9
Type the name of the text or sentence to view it.
Type: 'texts()' or 'sents()' to list the materials.
text1: Moby Dick by Herman Melville 1851
text2: Sense and Sensibility by Jane Austen 1811
text3: The Book of Genesis
text4: Inaugural Address Corpus
text5: Chat Corpus
text6: Monty Python and the Holy Grail
text7: Wall Street Journal
text8: Personals Corpus
text9: The Man Who Was Thursday by G . K . Chesterton 1908
```

gutenberg 데이터

1. NLTK 자연어 처리 패키지

```
1 nltk.corpus.gutenberg.fileids()
```

```
[ 'austen-emma.txt',
  'austen-persuasion.txt',
  'austen-sense.txt',
  'bible-kjv.txt',
  'blake-poems.txt',
  'bryant-stories.txt',
  'burgess-busterbrown.txt',
  'carroll-alice.txt',
  'chesterton-ball.txt',
  'chesterton-brown.txt',
  'chesterton-thursday.txt',
  'edgeworth-parents.txt',
  'melville-moby_dick.txt',
  'milton-paradise.txt',
  'shakespeare-caesar.txt',
  'shakespeare-hamlet.txt',
  'shakespeare-macbeth.txt',
  'whitman-leaves.txt' ]
```



austen-emma.txt

1. NLTK 자연어 처리 패키지

제인 오스틴(Jane Austen, 1775.12.16.-1817.7.18., 영국의 소설가)의 소설 엠마

```
1 emma = nltk.corpus.gutenberg.raw("austen-emma.txt")
```

```
1 print(emma[:200])
```

[Emma by Jane Austen 1816]

VOLUME I

CHAPTER I

Emma Woodhouse, handsome, clever, and rich, with a comfortable home
and happy disposition, seemed to unite some of the best blessings
of existence; an

토큰 생성(Tokenizing)

1. NLTK 자연어 처리 패키지

`sent_tokenize()` 함수는 문자열을 입력받아 문장 단위의 토큰 리스트를 출력

```
1 from nltk.tokenize import sent_tokenize
```

```
1 print(sent_tokenize(emma[:1000])[3])
```

Sixteen years had Miss Taylor been in Mr. Woodhouse's family,
less as a governess than a friend, very fond of both daughters,
but particularly of Emma.

토큰 생성(Tokenizing)

1. NLTK 자연어 처리 패키지

word_tokenize() 함수는 문자열을 입력받아 단어 단위의 토큰 리스트를 출력

```
1 from nltk.tokenize import word_tokenize
```

```
1 word_tokenize(emma[50:100])
```

```
['Emma',  
 'Woodhouse',  
 ',',  
 ',',  
 'handsome',  
 ',',  
 ',',  
 'clever',  
 ',',  
 ',',  
 'and',  
 'rich',  
 ',',  
 ',',  
 'with',  
 'a']
```

RegexTokenizer

1. NLTK 자연어 처리 패키지

RegexTokenizer 클래스를 이용하면 토큰화에 포함할 문자들을 정규표현식을 이용

```
1 from nltk.tokenize import RegexTokenizer
```

```
1 ret = RegexTokenizer("[\w]+")
```

```
1 ret.tokenize(emma[50:100])
```

```
['Emma', 'Woodhouse', 'handsome', 'clever', 'and', 'rich', 'with', 'a']
```

```
1 words = ["sending", "cooking", "flies", "lives",
2         "crying", "dying"]
```

형태소 분석

1. NLTK 자연어 처리 패키지

형태소(morpheme)는 언어학에서 일정한 의미가 있는 가장 작은 말의 단위를 의미

형태소 분석(morphological analysis)이란 단어로부터 어근, 접두사, 접미사, 품사 등 다양한 언어적 속성을 파악하고 이를 이용하여 형태소를 찾아내거나 처리하는 작업

형태소 분석의 예

- 어간 추출(Stemming)
- 원형 복원(Lemmatizing)
- 품사 부착(Part-Of-Speech Tagging)

PorterStemmer

1. NLTK 자연어 처리 패키지

```
1 words = ["sending", "cooking", "flies", "lives",  
2         "crying", "dying"]
```

```
1 from nltk.stem import PorterStemmer
```

```
1 pst = PorterStemmer()
```

```
1 pst.stem(words[0])
```

'send'

```
1 [pst.stem(w) for w in words]
```

['send', 'cook', 'fli', 'live', 'cri', 'die']

LancasterStemmer

1. NLTK 자연어 처리 패키지

```
1 words = ["sending", "cooking", "flies", "lives",  
2         "crying", "dying"]
```

```
1 from nltk.stem import LancasterStemmer
```

```
1 lst = LancasterStemmer()
```

```
1 [lst.stem(w) for w in words]
```

```
['send', 'cook', 'fli', 'liv', 'cry', 'dying']
```

RegexStemmer

1. NLTK 자연어 처리 패키지

```
1 words = ["sending", "cooking", "flies", "lives",
2          "crying", "dying"]
```

```
1 from nltk.stem.regex import RegexStemmer
```

```
1 rest = RegexStemmer('ing')
```

```
1 [rest.stem(w) for w in words]
```

```
['send', 'cook', 'flies', 'lives', 'cry', 'dy']
```

1. NLTK 자연어 처리 패키지

영어 외에 13개 국가의 언어에 대한 어간 추출을 지원

```
1 words2 = ['enviar', 'cocina', 'moscas', 'vidas',  
2          'llorar', 'morir']
```

```
1 from nltk.stem.snowball import SnowballStemmer
```

```
1 sbst = SnowballStemmer('spanish')
```

```
1 [sbst.stem(w) for w in words2]
```

```
['envi', 'cocin', 'mosc', 'vid', 'llor', 'mor']
```

사용 가능한 언어 : Arabic, Danish, Dutch, English, Finnish, French, German, Hungarian, Italian, Norwegian, Portuguese, Romanian, Russian, Spanish, Swedish

1. NLTK 자연어 처리 패키지

원형 복원(Lemmatizing은 어간 추출과 달리 원형 복원 후에도 그 단어는 같은 의미를 가짐

```
1 words3 = ["cooking", "believes"]
```

```
1 from nltk.stem.wordnet import WordNetLemmatizer
```

```
1 wl = WordNetLemmatizer()
```

```
1 [wl.lemmatize(w) for w in words3]
```

```
['cooking', 'belief']
```

```
1 [wl.lemmatize(w, pos='v') for w in words3]
```

```
['cook', 'believe']
```

품사 태깅(POS Tagging)

1. NLTK 자연어 처리 패키지

<https://dbrang.tistory.com/1139>

```
1 from nltk.tag import pos_tag
```

```
1 tagged_list = pos_tag(word_tokenize(sentence))
```

```
1 print(tagged_list)
```

```
[('Emma', 'NNP'), ('Woodhouse', 'NNP'), (',', ','), ('hand', 'NN'), ('some', 'NN'), (',', ','), ('clever', 'NN'), (',', ','), ('and', 'CC'), ('rich', 'JJ'), (',', ','), ('with', 'IN'), ('a', 'DT'), ('comfortable', 'JJ'), ('home', 'NN'), ('and', 'CC'), ('happy', 'JJ'), ('disposition', 'NN'), (',', ','), ('seemed', 'VBD'), ('to', 'TO'), ('unite', 'VB'), ('some', 'DT'), ('of', 'IN'), ('the', 'DT'), ('best', 'JJ'), ('S'), ('blessings', 'NNS'), ('of', 'IN'), ('existence', 'N
```

품사 태깅(POS Tagging)

1. NLTK 자연어 처리 패키지

```
1 nouns_list = [ t[0] for t in tagged_list if t[1]=="NN"]
```

```
1 nouns_list
```

```
['handsome', 'clever', 'home', 'disposition', 'existence', 'world']
```

품사 태깅(POS Tagging)

1. NLTK 자연어 처리 패키지

```
1 import re
```

```
1 pattern = re.compile('NN?')
```

```
1 nouns_list = [t[0] for t in tagged_list  
2               if pattern.match(t[1])]
```

```
1 nouns_list
```

```
['Emma',  
 'Woodhouse',  
 'handsome',  
 'clever',  
 'home',  
 'disposition',  
 'blessings',  
 'existence',  
 'years',  
 'world']
```

Text 클래스

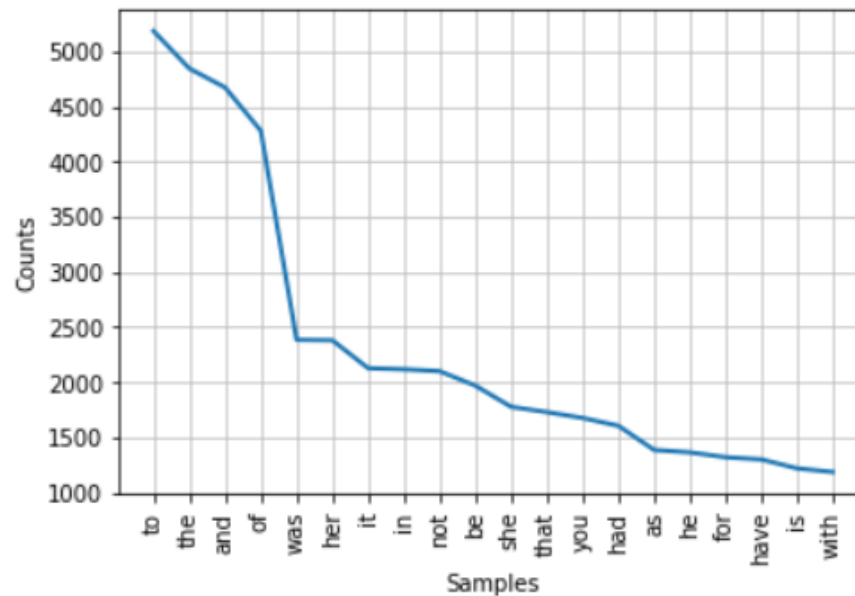
1. NLTK 자연어 처리 패키지

```
1 ret = RegexpTokenizer("[\w]{2,}")
```

```
1 from nltk import Text
```

```
1 emma_text = Text(ret.tokenize(emma))
```

```
1 emma_text.plot(20)
```



Text 클래스

1. NLTK 자연어 처리 패키지

```
1 emma_text.concordance('Emma', lines=5)
```

Displaying 5 of 865 matches:

```
Emma by Jane Austen 1816 VOLUME CHAPTER E
a by Jane Austen 1816 VOLUME CHAPTER Emma Woodhouse handso
me clever and rich w
f both daughters but particularly of Emma Between _them_ i
t was more the intim
nd friend very mutually attached and Emma doing just what
she liked highly est
by her own The real evils indeed of Emma situation were t
he power of having r
```

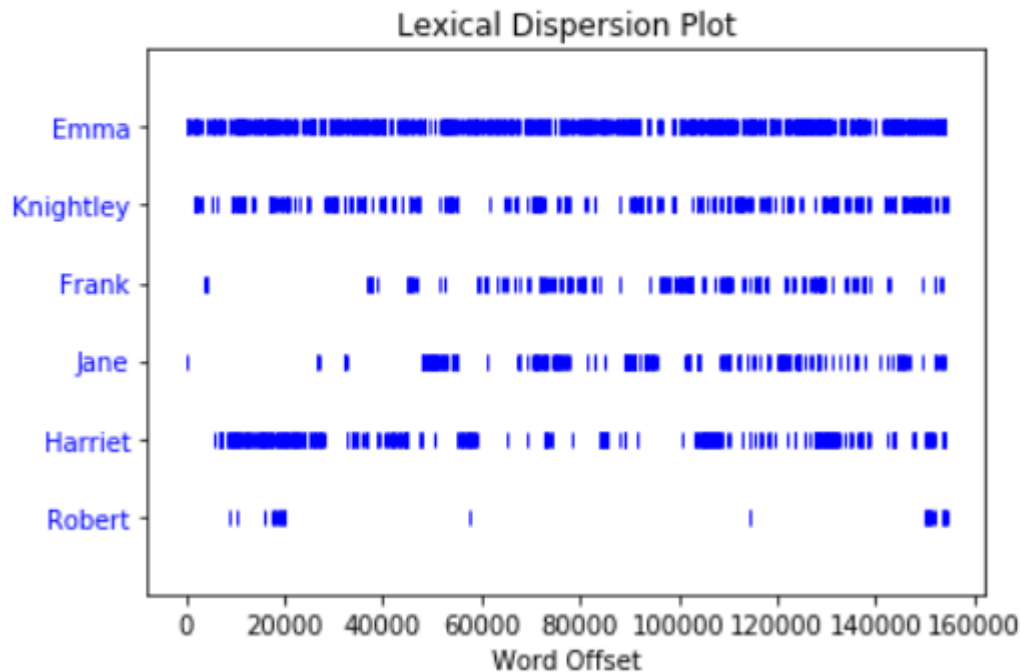
```
1 emma_text.similar("general")
```

```
love short it london time fact her which highbury and his
own well him
my hurry return vain person despair
```

1. NLTK 자연어 처리 패키지

지정한 단어가 얼마나 많이 나타날지 단어의 위치를 표시

```
1 emma_text.dispersion_plot(["Emma", "Knightley", "Frank",  
2                             "Jane", "Harriet", "Robert"])
```



FreqDist 클래스

1. NLTK 자연어 처리 패키지

문서에 사용된 단어(토큰)의 사용빈도 정보를 담은 클래스

```
1 from nltk import FreqDist
```

```
1 stopwords = ["Mr.", "Mrs.", "Miss", "Mr", "Mrs", "Dear"]
```

```
1 emma_tokens = pos_tag(emma_text)
```

```
1 names_list = [t[0] for t in emma_tokens  
2               if t[1]=="NNP" and t[0] not in stopwords]
```

```
1 emma_fd_names = FreqDist(names_list)
```

```
1 emma_fd_names
```

```
FreqDist({'Emma': 833, 'Harriet': 489, 'Weston': 439, 'Knightley': 389, 'Elton': 385, 'Woodhouse': 305, 'Jane': 298, 'Fairfax': 241, 'Churchill': 223, 'Frank': 208, ...})
```

2. 한글 형태소 분석



자연어 처리

2. 한글 형태소 분석

자연어 처리 (Natural Language Processing, NLP)

자연어 : 사람들이 일상적으로 사용하는 언어

인공어 : Artificial Language, 사람들이 필요에 의해서 만든 언어

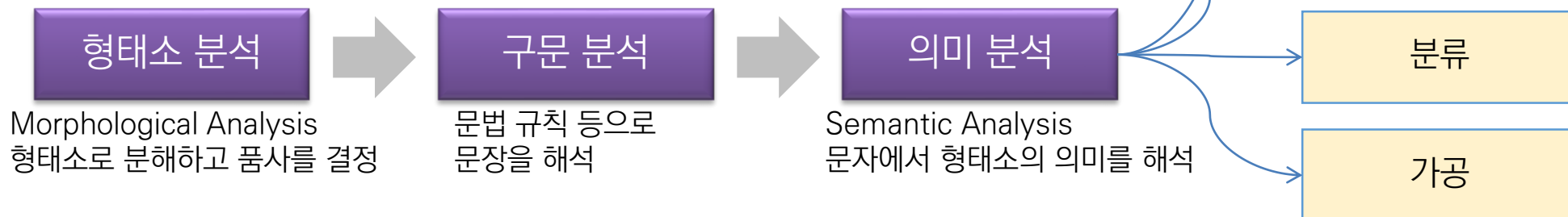
자연어 처리 분야

자연어 이해 : 형태소 분석 -> 의미 분석 -> 대화 분석

자연어 생성 : 대화 분석기 -> 문장 생성기 (생성 사전) -> 출력 문장

활용 분야 : 맞춤법 검사, 번역기, 검색 엔진, 키워드 분석 등

문서 (Document) -> 문단 (Paragraph) -> 문장 (Sentence) ->
어절 (Word phrase) -> 형태소 (Morpheme) -> 음절 (Syllable) ->
음소 (Phoneme)



자연어 처리 용어

2. 한글 형태소 분석

용어	상세
형태소(Morpheme)	의미를 가진 최소 단위입니다.
용언	꾸미는 말(동사, 형용사)입니다. 용언은 어근 + 어미로 구성됩니다.
어근(Stem)	용언이 활용할 때, 원칙적으로 모양이 변하지 않는 부분입니다.
어미	용언이 활용할 때, 변하는 부분으로 문법적 기능 수행합니다. 어미에는 연결 어미, 선어말 어미, 종결 어미가 있습니다.
자모	문자 체계의 한 요소(자음, 모음)입니다.
품사	명사, 대명사, 수사, 동사, 형용사, 관형사, 부사, 감탄사, 조사가 있습니다.
어절 분류	명사 + 주격 조사, 명사 + 목적격 조사, 명사 + 관형격 조사, 동사 + 연결 어미 또는 동사 + 선어말 어미 + 종결 어미 등으로 분류합니다.
불용어(Stopword)	검색 등에서 의미가 없어 무시되도록 설정된 단어들입니다.
n-gram	문자의 빈도와 문자간 관계를 의미합니다. "안녕하세요"를 2-gram으로 나누면, "안녕", "녕하", "하세", "세요"로 나눌 수 있습니다.

형태소

2. 한글 형태소 분석



형태소

- ▶ 의미를 가진 최소의 단위
- ▶ 문법적, 관계적인 뜻을 나타내는 단어 또는 단어의 부분
- ▶ 체언 (명사, 대명사, 수사), 수식언 (관형사, 부사), 독립언 (감탄사), 관계언 (조사, 서술격조사), 용언 (동사, 형용사, 조동보조어간), 부속어 (어미, 접미)



형태소의 종류

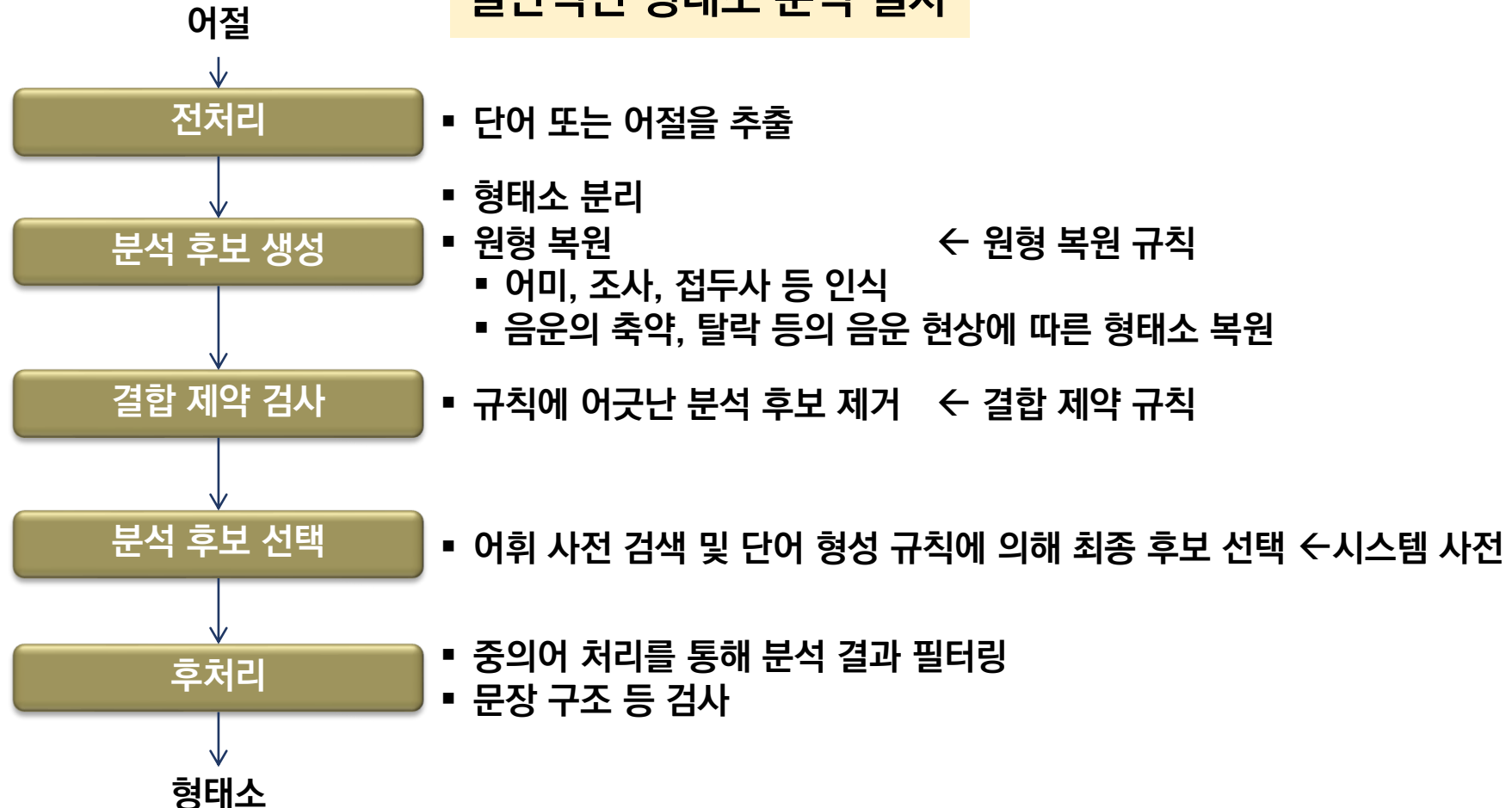
- ▶ 실질 형태소
 - 체언, 수식언, 감탄사, 용언의 어근
 - 구체적인 대상, 동작, 상태 등을 나타내는 형태소로 어휘 형태소
 - 검색 엔진에서 색인의 대상이 됩니다.
- ▶ 형식 형태소
 - 조사, 어말어미, 접미사, 접두사, 선어말어미
 - 형태소 간의 관계를 나타내는 형태소로 문법 형태소

형태소 분석

2. 한글 형태소 분석

단어 또는 어절을 구성하는 각 형태소 분리하고 형태소의 기본형(어근) 및 품사 정보 추출

일반적인 형태소 분석 절차



형태소 분석 엔진

2. 한글 형태소 분석

KoNLPy

- Python용 자연어 처리기(Korean NLP in Python)(GPL v3+라이선스)
- <http://konlpy.readthedocs.org/>

KOMORAN

- 자바로 만든 형태소 분석기(Apache v2 라이선스)
- <https://shineware.tistory.com/tag/KOMORAN/>

HanNanum

- 자바로 만든 형태소 분석기(GPL v3 라이선스)
- <http://semanticweb.kaist.ac.kr/home/index.php/HanNanum>

KoNLP

- R용 자연어 처리기(Korean NLP)(GPL v3 라이선스)
- <https://github.com/haven-jeon/KoNLP>

KoNLPy 패키지

2. 한글 형태소 분석

한국어 정보처리를 위한 파이썬 패키지

오픈 소스 소프트웨어이며 [GPL v3 이상] 라이선스로 배포

공식 사이트

- <http://konlpy.org/>
- <https://github.com/konlpy/konlpy>

KoNLPy 패키지 설치

- (base) C:\Users\COM>pip install konlpy
- KoNLPy는 JType1 패키지에 의존

JDK 설치(1.7 이상)

- <http://jdk.java.net/14/> -> Builds -> Windows/x64 zip 다운로드 후 압축 풀기
- JAVA_HOME 환경변수 설정

형태소 분석

2. 한글 형태소 분석

형태소를 비롯하여, 어근, 접두사/접미사, 품사(POS, part-of-speech) 등 다양한 언어적 속성의 구조를 파악하는 것

품사 태그를 알아야 함

- <https://konlpy-ko.readthedocs.io/ko/v0.4.3/morph/#comparison-between-pos-tagging-classes>

품사 태깅

- 형태소의 뜻과 문맥을 고려하여 그것에 마크업을 하는 일
- 단어와 /(슬래시) 뒤에 품사가 표시되어 나누어지도록 하는 것
예)

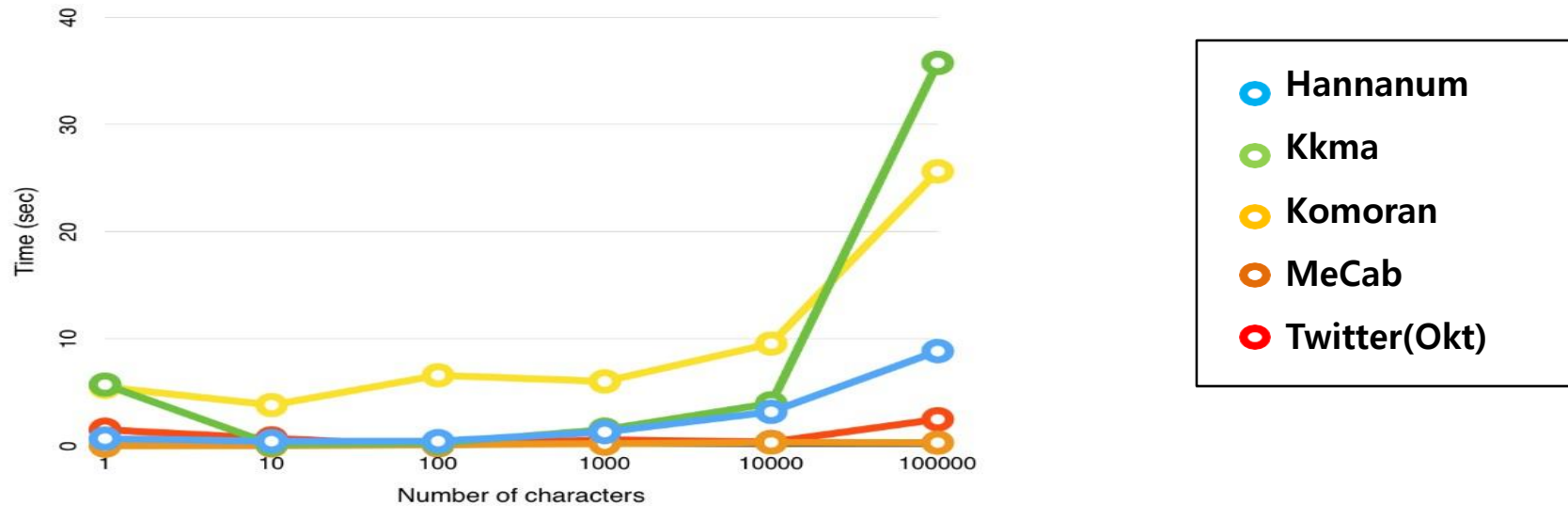
원문 : 가방에 들어가신다

품사 태깅 : 가방/NNG + 에/JKM + 들어가/VV + 시/EPH + 다/EFN

형태소 분석 모듈들

성능비교 참조 : <https://konlpy-ko.readthedocs.io/ko/v0.4.3/morph/#pos-tagging-with-konlpy>

- KoNLPy 라이브러리에는 형태소 분석을 위해 지원하는 클래스가 여러 개 있음



코모란(Komoran)의 품사 태그와 설명

2. 한글 형태소 분석

Komoran(ntags=42)

태그	설명	태그	설명	태그	설명
NNG	일반 명사	JKC	보격 조사	XSV	동사 파생 접미사
NNP	고유 명사	JKG	관형격 조사	XSA	형용사 파생 접미사
NNB	의존 명사	JKO	목적격 조사	XR	어근
NR	수사	JKB	부사격 조사	SF	마침표, 물음표, 느낌표
NP	대명사	JKV	호격 조사	SE	줄임표
VV	동사	JKQ	인용격 조사	SS	따옴표, 괄호표, 줄표
VA	형용사	JC	접속 조사	SP	쉼표, 가운데점, 콜론, 빗금
VX	보조 용언	JX	보조사	SO	붙임표(물결, 숨김, 빠짐)
VCP	긍정 지정사	EP	선어말어미	SW	기타기호 (논리수학기호, 화폐기호)
VCN	부정 지정사	EF	종결 어미	SH	한자
MM	관형사	EC	연결 어미	SL	외국어
MAG	일반 부사	ETN	명사형 전성 어미	SN	숫자
MAJ	접속 부사	ETM	관형형 전성 어미	NF	명사추정범주
IC	감탄사	XPN	체언 접두사	NV	용언추정범주
JKS	주격 조사	XSN	명사파생 접미사	NA	분석불능범주

<https://konlpy-ko.readthedocs.io/ko/v0.4.3/morph/#comparison-between-pos-tagging-classes>

2. 한글 형태소 분석

카이스트 SWRC에 의해 개발된 자바로 만들어진 형태소 분석기

`konlpy.tag.Hannanum(jvmpath=None)`

메서드	설명
<code>analyze(phrase)</code>	각 토큰에 대한 다양한 형태소 후보를 반환합니다.
<code>morphs(phrase)</code>	형태소 분석 문구를 반환합니다.
<code>nouns(phrase)</code>	명사를 추출합니다.
<code>pos(phrase, flatten=True)</code>	<code>ntags</code> 는 태그의 수. 9(한 문자) 또는 22(두 문자)일 수 있습니다. <code>flatten</code> 이 <code>False</code> 이면 어절을 보존합니다.

2. 한글 형태소 분석

한나눔 형태소 분석기를 이용해 태깅하는 예

```
1 text = """아름답지만 다소 복잡하기도한 한국어는 전세계에서 13번째로 많이
2 사용되는 언어입니다."""
```

```
1 from konlpy.tag import Hannanum
2 han = Hannanum()
```

```
1 print(han.analyze(text))
```

```
[[('아름답', 'paa'), ('지만', 'ecs')], [('아름답', 'paa'), ('지', 'ecs'), ('만', 'jxc'), ('다소', 'mag')], [('다소', 'ncn')], [('복잡하기도한', 'ncn')], [('복잡하기도한', 'nqq')], [('한국어', 'ncn'), ('는', 'jxc')], [('전세계', 'ncn'), ('에서', 'jca')], [('전세계', 'ncn'), ('에서', 'jca')], [('전', 'xp'), ('세계', 'ncn'), ('에서', 'jca')], [('13', 'nnc'), ('번', 'nbu'), ('째', 'xsnu'), ('로', 'jca')], [('많', 'paa'), ('이', 'xsa')], [('많이', 'mag')], [], [('사용', 'ncpa'), ('되', 'xsvn'), ('는', 'etm')], [('언어', 'ncn'), ('이', 'jp'), ('입니다', 'ef')], [('.', 'sf')], [('.', 'sy')]]
```

```
1 print(han.morphs(text))
```

```
['아름답', '지만', '다소', '복잡하기도한', '한국어', '는', '전세계', '에서', '13번', '째', '로', '많', '이', '사용', '되', '는', '언어', '이', '입니다', '.']
```

```
1 print(han.nouns(text))
```

```
['복잡하기도한', '한국어', '전세계', '13번', '사용', '언어']
```

2. 한글 형태소 분석

 서울대학교 IDS(Intelligent Data Systems) 연구실 konlpy.tag.Kkma(jvmpath=None)

메서드	설명
morphs(phrase)	형태소 분석 문구를 반환합니다.
nouns(phrase)	명사를 추출합니다.
pos(phrase, flatten=True)	flatten이 False이면 어절을 보존합니다.

2. 한글 형태소 분석

 꼬꼬마 형태소 분석기를 이용해 태깅하는 예

```
1 from konlpy.tag import Kkma
2 kkma = Kkma()
```

```
1 print(kkma.morphs(text))
```

['아름답', '지만', '다소', '복잡', '하', '기', '도', '한', '한국어', '는', '전세계', '에서', '13', '번째', '로', '많이', '사용', '되', '는', '언어', '이', '됩니다', '.']

```
1 print(kkma.nouns(text))
```

['복잡', '한국어', '전세계', '13', '13번째', '번째', '사용', '언어']

```
1 print(kkma.pos(text))
```

[('아름답', 'VA'), ('지만', 'ECE'), ('다소', 'MAG'), ('복잡', 'NNG'), ('하', 'XSV'), ('기', 'ETN'), ('도', 'JX'), ('한', 'MDN'), ('한국어', 'NNG'), ('는', 'JX'), ('전세계', 'NNG'), ('에서', 'JKM'), ('13', 'NR'), ('번째', 'NNB'), ('로', 'JKM'), ('많이', 'MAG'), ('사용', 'NNG'), ('되', 'XSV'), ('는', 'ETD'), ('언어', 'NNG'), ('이', 'VCP'), ('됩니다', 'EFN'), ('.', 'SF')]

2. 한글 형태소 분석

2013년 Shineware에서 자바로 만든 오픈소스 한국어 형태소 분석기

`konlpy.tag.Komoran(jvmpath=None, dicpath=None)`

메서드	설명
<code>morphs(phrase)</code>	형태소 분석 문구를 반환합니다.
<code>nouns(phrase)</code>	명사를 추출합니다.
<code>pos(phrase, flatten=True)</code>	<code>flatten</code> 이 <code>False</code> 이면 어절을 보존합니다.

Komoran

2. 한글 형태소 분석

코모란 형태소 분석기를 이용해 태깅하는 예

```
1 from konlpy.tag import Komoran
2 kom = Komoran()
```

```
1 print(kom.morphs(text))
```

['아름답', '지만', '다소', '복잡', '하', '기', '도', '하', 'ㄴ', '한국어', '는', '전', '세계', '에서', '13', '번', '째', '로', '많이', '사용', '되', '는', '언어', '이', '입니다', '.']

```
1 print(kom.nouns(text))
```

['한국어', '전', '세계', '번', '사용', '언어']

```
1 print(kom.pos(text))
```

[('아름답', 'VA'), ('지만', 'EC'), ('다소', 'MAG'), ('복잡', 'XR'), ('하', 'XS A'), ('기', 'ETN'), ('도', 'JX'), ('하', 'VV'), ('ㄴ', 'ETM'), ('한국어', 'NNP'), ('는', 'JX'), ('전', 'NNG'), ('세계', 'NNG'), ('에서', 'JKB'), ('13', 'SN'), ('번', 'NNB'), ('째', 'XSN'), ('로', 'JKB'), ('많이', 'MAG'), ('사용', 'NNG'), ('되', 'XSV'), ('는', 'ETM'), ('언어', 'NNG'), ('이', 'VCP'), ('입니다', 'EF'), ('.', 'SF')]

2. 한글 형태소 분석

컴퓨터를 이용해 자연어 분석 작업을 할 수 있도록 만든 문서 집합

KoNLPy 패키지를 설치하고 사용할 수 있는 말뭉치

- kolaw
 - 한국 법률 말뭉치
 - constitution.txt
- kobill
 - 대한민국 국회 의안 말뭉치
 - 파일 ID는 의안 번호를 의미
 - 1809890.txt - 1809899.txt

```
1 from konlpy.corpus import kolaw
2 c = kolaw.open('constitution.txt').read()
3 print(c[:100])
```

대한민국헌법

유구한 역사와 전통에 빛나는 우리 대한국민은 3·1운동으로 건립된 대한민국임시정부의 법통과 불의에 항거한 4·19민주이념을 계승하고, 조국의 민주개혁과 평화적 통일의

```
1 from konlpy.corpus import kobill
2 d = kobill.open('1809890.txt').read()
3 print(d[150:300])
```

초등학교 저학년의 경우에도 부모의 따뜻한 사랑과 보살핌이 필요한 나이이나, 현재 공무원이 자녀를 양육하기 위하여 육아휴직을 할 수 있는 자녀의 나이는 만 6세 이하로 되어 있어 초등학교 저학년인 자녀를 돌보기 위해서는 해당 부모님은 일자리를 그만 두어야

워드 클라우드

2. 한글 형태소 분석

단어를 출현 빈도에 비례하는 크기로 단어의 빈도수를 시각화하는 기법

빈도분석이 된 데이터를 시각화하기 위해 워드클라우드를 사용

pip install wordcloud

```
1 from konlpy.corpus import kolaw
2 data = kolaw.open('constitution.txt').read()
```

```
1 from konlpy.tag import Komoran
2 komoran = Komoran()
```

```
1 print(komoran.nouns("%r"%data[0:1000]))
```

['대한민국', '헌법', '유구', '한', '사', '전통', '국민', '운동', '거
립', '대한민국', '임시', '정부', '법
'계승', '조국', '민주개혁', '평화',
도', '동포애', '민족', '단결', '사회
'조화', '바탕', '자유', '민주', '기본
'문화', '영역', '각인', '기회', '능력
'권리', '책임', '의무', '완수', '안', '국민', '생활', '관공', '양성',
'밖', '항구', '세계', '평화', '인류', '공영', '이바지', '우리들의', '자
소', '아저', '자유', '행복', '화부', '거', '다짐', '녀', '7월 12일', '제

헌법 말뭉치를 불러와 형태소
분석 후 명사만 추출

샘플을 이용한 워드 클라우드 그리기

2. 한글 형태소 분석

```
1 word_list = komoran.nouns("%r"%data[0:1000])
```

```
1 text = ' '.join(word_list)
```

```
1 text
```

명사들을 공백으로 이어서
하나의 문장으로 만듦

'대한민국 헌법 유구 한 역사 전통 국민 운동 건립 대한민국 임시 정부 법통 불의 항거 민주 이념 계승 조국 민주개혁 평화 통일 사명 입각 정의 인도 동 포애 민족 단결 사회 폐습 불의 타파 자율 조화 바탕 자유 민주 기본 질서 정치 경제 사회 문화 영역 각인 기회 능력 최고 도로 발취 자유 권리 책임 의무 완수 안 국민 생활 균등 향상 밖 항구 세계 평화 인류 공영 이바지 우리들의 자손 안전 자유 행복 확보 것 다짐 년 7월 12일 제정 차 개정 헌법 국회 의결 국민 투표 개정 장 강 대한민국 민주공화국 대한민국 주권 국민 권력 국민 대한민국 국민 요건 법률 국가 법률 바 재외국민 보호 의무 대한민국 영토 한반도 부속 도서 대한민국 통일 지향 자유 민주 기본 질서 입각 평화 통일 정책 수립 추진 대한민국 국제 평화 유지 노력 침략 전쟁 부인 국군 국가 안전 보장 국토방위 신성 의무 수행 사명 정치 중립 준수 헌법 체결 공포 조약 일반 승인 국제 법규 국내법 효력 외국인 국제법 조약 바 지위 보장 공무원 국민 전체 봉사자 국민 책임 공무원 신분 정치 중립 법률 바 보장 정당 설립 자유 복수 정당'

샘플을 이용한 워드 클라우드 그리기

2. 한글 형태소 분석

```
1 import matplotlib.pyplot as plt
2 %matplotlib inline
3 from wordcloud import WordCloud
```

```
1 wordc = WordCloud()
2 wordc.generate(text)
```

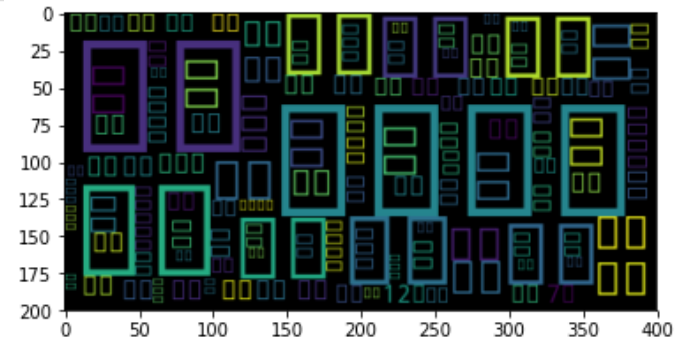
<wordcloud.wordcloud.WordCloud at 0x1d532a20>

```
1 plt.figure()
2 plt.imshow(wordc, interpolation="bilinear")
```

<matplotlib.image.AxesImage at 0x1f2da6d8>



워드클라우드를 그렸지만
한글이 깨짐



한글 처리

2. 한글 형태소 분석

```
1 wordc = WordCloud(background_color='white', max_words=20,
2                  font_path='c:/Windows/Fonts/malgun.ttf',
3                  relative_scaling=0.2)
```

```
1 wordc.generate(text)
```

<wordcloud.wordcloud.WordCloud at 0x1d536cc0>

```
1 plt.figure()
2 plt.imshow(wordc, interpolation="bilinear")
3 plt.axis('off')
```



워드클라우드에 한글을
표시하기 위해
워드클라우드 객체를
생성할 때 폰트를 지정



전체 데이터를 이용한 워드클라우드 생성

2. 한글 형태소 분석

<https://matplotlib.org/stable/users/explain/colors/colormaps.html>

```
1 word_list = komoran.nouns("%r"%data)
2 text = ' '.join(word_list)
```

```
1 wordcloud = WordCloud(background_color='white', max_words=2000,
2 font_path='c:/Windows/Fonts/malgun.ttf',
3 relative_scaling=0.2)
```

```
1 wordcloud.generate(text)
```

<wordcloud.wordcloud.WordCloud at 0x203626a0>

```
1 plt.figure(figsize=(15, 10))
2 plt.imshow(wordcloud, interpolation="bilinear")
3 plt.axis('off')
```



2. 한글 형태소 분석

```
1 from wordcloud import STOPWORDS
2 from sklearn.feature_extraction.stop_words import ENGLISH_STOP_WORDS
3
4 불용어 = STOPWORDS | ENGLISH_STOP_WORDS | set(["대통령", "국가"])
```

```
1 wordcloud = WordCloud(background_color='white', max_words=2000,  
2 stopwords=불용어,  
3 font_path='c:/Windows/Fonts/malgun.ttf',  
4 relative_scaling=0.2)  
5 wordcloud.generate(text)
```

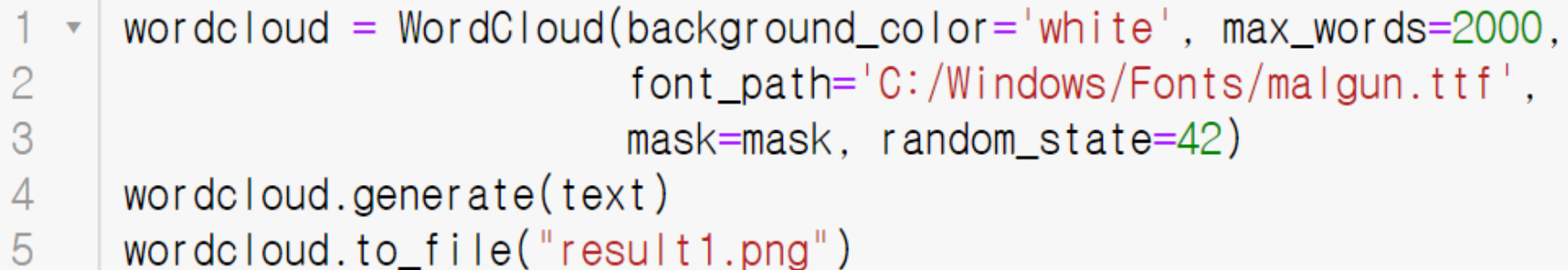
```
<wordcloud.wordcloud.WordCloud at 0x204a2f28>
```

```
1 plt.figure(figsize=(15, 10))
2 plt.imshow(wordcloud, interpolation="bilinear")
3 plt.axis('off')
```



“대통령”, “국가” 단어를 불용어로 지정한 후 결과

```
1 from PIL import Image
2 import numpy as np
3 img = Image.open("south_korea.png").convert("RGBA")
4 mask = Image.new("RGB", img.size, (255,255,255))
5 mask.paste(img, img)
6 mask = np.array(mask)
```



```
1 import nltk
2 import matplotlib.font_manager as fm
3 plt.figure(figsize=(12,6))
4 font_name = fm.FontProperties(fname="C:/Windows/Fonts/H2GTRM.TTF").get_name()
5 plt.rc("font", family=font_name)
6 nltk.Text(word_list).plot(50)
```



워드 임베딩(Word Embedding)

2. 한글 형태소 분석

- 워드 임베딩(Word Embedding)은 단어를 벡터로 표현하는 대표적인 방법으로 주로 희소 표현에서 밀집 표현으로 변환하는 것을 의미
- 희소 표현(Sparse Representation)
 - 원-핫 인코딩을 통해서 나온 원-핫 벡터들은 표현하고자 하는 단어의 인덱스의 값만 1이고, 나머지 인덱스에는 전부 0으로 표현되는 벡터 표현 방법
 - 벡터 또는 행렬(Matrix)의 값이 대부분이 0으로 표현되는 방법
 - 원-핫 벡터는 희소 벡터(Sparse Vector)
- 밀집 표현(Dense Representation)
 - 사용자가 설정한 값으로 모든 단어의 벡터 표현의 차원을 맞춤
 - 이 과정에서 더 이상 0과 1만 가진 값이 아니라 실수값을 가지게 됨
 - 벡터의 차원이 조밀해졌다고 하여 밀집 벡터(Dense Vector)라고 함
- 워드 임베딩(Word Embedding)
 - 단어를 밀집 벡터(Dense Vector)의 형태로 표현하는 방법
 - 이 밀집 벡터를 워드 임베딩 과정을 통해 나온 결과라고 하여 임베딩 벡터(Embedding Vector)이라고 함

Word2Vec

2. 한글 형태소 분석

- 단어 간 유사성을 고려하기 위해서는 단어의 의미를 벡터화하기 위한 패키지
- 희소 표현(Sparse Representation)
 - 단어간 유사성을 표현할 수 없다는 단점
- 분산 표현(Distributed Representation)
 - 단어의 '의미'를 다차원 공간에 벡터화하는 방법
 - '비슷한 위치에서 등장하는 단어들은 비슷한 의미를 가진다'라는 가정
 - 분산 표현을 이용하여 단어의 의미를 벡터화하는 작업은 워드 임베딩(Embedding) 작업에 속함

Word2Vec

2. 한글 형태소 분석

- CBOW(Continuous Bag of Words)
 - 주변에 있는 단어들을 가지고, 중간에 있는 단어들을 예측하는 방법
- Skip-Gram
 - 중간에 있는 단어로 주변 단어들을 예측하는 방법
- ***class Word2Vec(sentences=None, size=100, window=5, min_count=5, workers=3)***
 - size : 특징 벡터의 차원
 - window : 문장 내 현재 단어와 예측 단어 사이의 최대 거리
 - min_count : 총 빈도가 이보다 낮은 모든 단어를 무시함
 - workers : 모델을 훈련시킬 쓰레드의 수
 - <https://www.pydoc.io/pypi/gensim-3.2.0/autoapi/models/word2vec/index.html#models.word2vec.Word2Vec>
- 패키지 설치
 - pip install gensim

뉴스기사 워드 임베딩 하기

2. 한글 형태소 분석

```
1 import requests
2 rss_url = "http://fs.jtbc.joins.com/RSS/economy.xml"
3 jtbc_economy = requests.get(rss_url)
```

```
1 from bs4 import BeautifulSoup
2 economy_news_list = BeautifulSoup(jtbc_economy.content, "xml")
3 link_list = economy_news_list.select("item > link")
```

```
1 from konlpy.tag import Kkma
2 kkma = Kkma()
```

```
1 news = [] # transaction
2 for link in link_list:
3     news_url = link.text
4     news_response = requests.get(news_url)
5     news_soup = BeautifulSoup(news_response.content, "html.parser")
6     news_content = news_soup.select_one("#articlebody > .article_content")
7     news.append(list(filter(lambda word: len(word)>1,
8                             kkma.nouns(news_content.text))))
```

뉴스기사 워드 임베딩 하기

2. 한글 형태소 분석

```
1 from gensim.models import Word2Vec
2 model = Word2Vec(news, size=100, window=5, min_count=2, workers=-1)
```

```
1 model.wv.most_similar("누진제")
```

```
[('이사회', 0.2872373163700104),
 ('신도시', 0.2726539075374603),
 ('협약체', 0.2622441053390503),
 ('투플러스', 0.23011885583400726),
 ('지방', 0.22897236049175262),
 ('전기', 0.2165725827217102),
 ('홍남기', 0.21561753749847412),
 ('수법', 0.20883645117282867),
 ('국토교통부', 0.20025959610939026),
 ('50만원', 0.19682154059410095)]
```

3. 연관분석



오렌지주스를 구매하는 사람은 와인을 구매할까?

- 1 소주, 콜라, 와인
- 2 소주, 오렌지주스, 콜라
- 3 콜라, 맥주, 와인
- 4 소주, 콜라, 맥주
- 5 오렌지주스, 와인

3.1 연관 분석(Association Analysis) 개요

1.1. 연관 분석

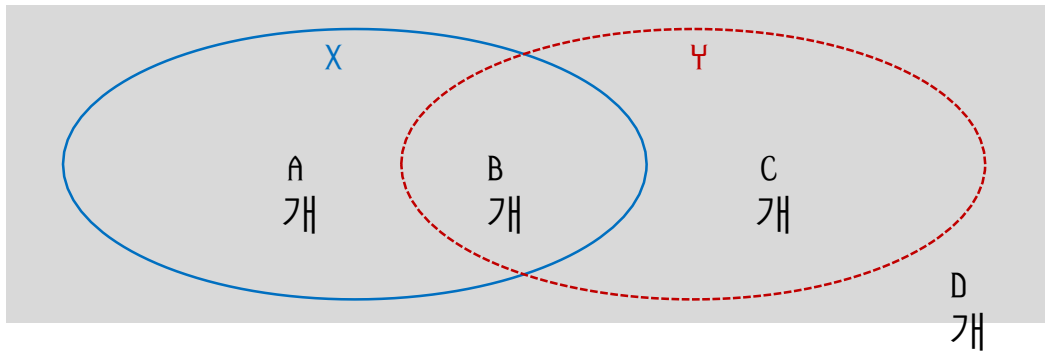
1절. 연관 분석(Association Analysis) 개요

- 연관 분석(Association Analysis)은 데이터들 사이에서 ‘자주 발생하는 속성을 찾는’ 그리고 그 속성들 사이에 ‘연관성이 어느 정도 있는지’를 분석하는 방법
- 연관 규칙을 찾는 알고리즘으로는 apriori, FP-growth, DHP 알고리즘 등이 있음
- apriori 알고리즘이 비교적 구현이 간단하고 성능 또한 높음
- `pip install apyori`

1.2. 연관 분석 평가

1절. 연관 분석(Association Analysis) 개요

- 지지도(Support)는 전체 트랜잭션에서 항목 집합(X, Y)을 포함하는 트랜잭션의 비율을 의미
- 신뢰도(Confidence)는 X를 포함하는 트랜잭션 중 Y도 포함하는 트랜잭션의 비율을 의미



범례 :  X를 포함하는 트랜잭션
 Y를 포함하는 트랜잭션

지지도 = $B \text{ 개} / (A \text{ 개} + B \text{ 개} + C \text{ 개} + D \text{ 개})$

신뢰도 = $B \text{ 개} / (A \text{ 개} + B \text{ 개})$

연관 분석 평가 척도

1절. 연관 분석(Association Analysis) 개요

평가 척도	설명
지지도 (Support)	- 연관 규칙이 얼마나 중요한지에 대한 평가 - 낮은 지지도의 연관 규칙은 우연히 발생한 트랜잭션에서 생성된 규칙일 수 있음
신뢰도 (Confidence)	연관 규칙이 얼마나 믿을 수 있는지 평가
향상도 (Lift)	$Lift = P(Y X) / P(Y) = P(X \cap Y) / P(X)P(Y) = \text{신뢰도} / P(Y)$ $X \rightarrow Y$의 신뢰도 / Y의 지지도 1 보다 작으면 음의 상관관계(설사약과 변비약), 1이면 상관관계 없음(과자와 후추), 1 보다 크면 양의 상관관계(빵과 버터)
파이 계수 (Phi coefficient)	-1 : 완전 음의 상관관계, 0 : 상관관계 없음, 1 : 완전 양의 상관관계
IS 척도	비대칭적 이항 변수에 적합한 척도로 값이 클수록 상관관계가 높음
(Interest-Support)	$P(X, Y) / P(X) * P(Y)$

3.2 트랜잭션 데이터

2.1. CSV 파일로부터 트랜잭션 데이터 생성

2절. 트랜잭션 데이터

- 트랜잭션(Transaction)은 서로 관련 있는 데이터의 모음
- 연관분석 시 트랜잭션으로부터 연관 규칙을 도출
- 파이썬의 트랜잭션 데이터는 리스트의 리스트 형식

```
1 import csv
2 with open('basket.csv', 'r', encoding='UTF8') as cf:
3     transactions = []
4     r = csv.reader(cf)
5     for row in r:
6         transactions.append(row)
7
```

basket.csv

```
1 소주,콜라,와인
2 소주,오렌지주스,콜라
3 콜라,맥주,와인
4 소주,콜라,맥주
5 오렌지주스,와인
```

```
1 transactions
```

```
[['소주', '콜라', '와인'],
 ['소주', '오렌지주스', '콜라'],
 ['콜라', '맥주', '와인'],
 ['소주', '콜라', '맥주'],
 ['오렌지주스', '와인']]
```

3.3 연관 분석(Association Analysis)

3.1. 연관 규칙 생성

3절. 연관 분석(Association Analysis)

- **apriori() 연역적 알고리즘(a priori algorithm)을 사용하여 연관규칙 을 알아내는 함수**

```
1 from apyori import apriori
2 rules = apriori(transactions, min_support=0.1, min_confidence=0.1)
3 results = list(rules)
4 type(results)
```

list

```
1 results[0]
```

```
RelationRecord(items=frozenset({'맥주'}), support=0.4, ordered_statistics=[OrderedStatistic(items_base=frozenset(), items_add=frozenset({'맥주'}), confidence=0.4, lift=1.0)])
```

```
1 results[10]
```

```
RelationRecord(items=frozenset({'콜라', '소주'}), support=0.6, ordered_statistics=[OrderedStatistic(items_base=frozenset({'소주'}), items_add=frozenset({'콜라'}), confidence=1.0, lift=1.25), OrderedStatistic(items_base=frozenset({'콜라'}), items_add=frozenset({'소주'}), confidence=0.7499999999999999, lift=1.2499999999999998)])
```

이렇게 생성한 규칙은
namedtuple 형식으로
저장되어 있음

연관 규칙 형식

3절. 연관 분석(Association Analysis) > 3.1. 연관 규칙 생성

- 연관 규칙은 namedtuple 형식으로 저장되어 있기 때문에 원하는 값을 찾거나 출력하기 위해서는 연관 규칙 결과가 어떤 형식으로 저장되어 있는지 이해해야 함
- apyori.py 파일에 정의되어 있는 RelationRecord와 OrderStatistic은 다음처럼 namedtuple로 정의되어 있음

```
SupportRecord = namedtuple( 'SupportRecord', ('items', 'support'))
```

```
RelationRecord = namedtuple( 'RelationRecord', SupportRecord._fields + ('ordered_statistics',))
```

```
OrderedStatistic = namedtuple( 'OrderedStatistic', ('items_base', 'items_add', 'confidence', 'lift',))
```

```
C:\Users\사용자\Anaconda3\Lib\site-packages\apyori.py
```

3.2. 연관 규칙 조회

3절. 연관 분석(Association Analysis)

```
1 print("lhs Wtrhs WtWtsupportWtWtconfidenceWtlift")
2 for row in results:
3     support = row[1]
4     ordered_stat = row[2]
5     for ordered_item in ordered_stat:
6         lhs = [x for x in ordered_item[0]]
7         rhs = [x for x in ordered_item[1]]
8         confidence = ordered_item[2]
9         lift = ordered_item[3]
10    print(lhs, " => ", rhs, "Wt{:>5.4f}Wt{:>5.4f}Wt{:>5.4f}".W
11          format(support, confidence, lift))
12
```

lhs	rhs	support	confidence	lift
[] => ['맥주']		0.4000	0.4000	1.0000
[] => ['소주']		0.6000	0.6000	1.0000
[] => ['오렌지주스']		0.4000	0.4000	1.0000
[] => ['와인']		0.6000	0.6000	1.0000
[] => ['콜라']		0.8000	0.8000	1.0000
['맥주'] => ['소주']		0.2000	0.5000	0.8333
['소주'] => ['맥주']		0.2000	0.3333	0.8333
['맥주'] => ['와인']		0.2000	0.5000	0.8333

3.3. 연관 규칙 평가

3절. 연관 분석(Association Analysis)

- 오렌지주스를 구매한 사람은 와인을 구매할까?
- 앞의 결과에서 오렌지주스와 와인의 연관관계를 보면 다음과 같음
 - ['오렌지주스'] => ['와인'], 0.2000, 0.5000, 0.8
 - ['와인'] => ['오렌지주스'], 0.2000, 0.3333, 0.8
- 오렌지주스와 와인을 구매한 사람은 순서에 상관없이 전체의 20%
- 오렌지주스를 산 고객의 50%가 와인을 구매
- 향상도는 0.833 이고 향상도가 1보다 작은 것은 음의 상관관계를 의미
- 그러므로 오렌지주스를 구매한 사람은 와인을 구매하지 않음

3.4 뉴스 기사 연관 분석실습

4.1. 뉴스 RSS 서버에서 링크 주소 가져오기

4장. 뉴스 기사 연관 분석

- JTBC 경제분야 크롤링 후 연관 분석
- RSS 주소: <http://fs.jtbc.joins.com/RSS/economy.xml>

```
1 import requests
2 rss_url = "http://fs.jtbc.joins.com/RSS/economy.xml"
3 jtbc_economy = requests.get(rss_url)
```

```
1 from bs4 import BeautifulSoup
2 economy_news_list = BeautifulSoup(jtbc_economy.content, "xml")
3 link_list = economy_news_list.select("item > link")
```

```
1 len(link_list)
```

20

```
1 link_list[0].text
```

'http://news.jtbc.joins.com/article/article.aspx?news_id=NB11822526'

4.2. KoNLPy 패키지

4장. 뉴스 기사 연관 분석

- 한국어 정보처리를 위한 파이썬 패키지
- 오픈 소스 소프트웨어이며 [GPL v3 이상] 라이선스로 배포
- 공식 사이트는 다음과 같습니다.
 - <http://konlpy.org/en/latest/>
 - <https://github.com/konlpy/konlpy>
- KoNLPy 패키지 설치
 - KoNLPy는 JPytype1 패키지에 의존
 - (base) C:\WUsers\WCOM>pip install konlpy
- JDK를 설치해야 함
 - <http://jdk.java.net/12/> -> Builds -> Windows/x64 zip 다운로드
 - 압축 풀기
 - JAVA_HOME 환경변수 설정

4. 케라스를 이용한 인공지능경망 구현



인공지능의 역사 및 딥러닝의 혁신

3. 인공지능망

인공지능의 역사 및 딥러닝의 혁신

~1990년 : 이론 정립

~2000년 : 구현 시도

~2010년 : 본격 시도

2010년~ : 혁신의 시작
(딥러닝 기반의 인공지능)

시대별 한계 : - 컴퓨팅의 한계로 제안된 이론 구현의 어려움 - 데이터의 한계로 현실 문제 해결하지 못함 - 알고리즘의 한계로 완성도 부족

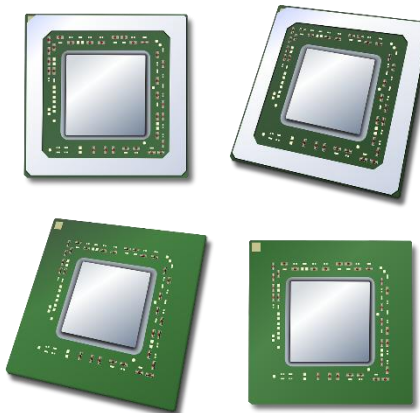
혁신적 알고리즘 제안



Geoffrey Hinton
(Univ. of Toronto,
Google)

- 혁신적 딥러닝 이론 제안(2006년)
- 실제 구현으로 혁신적 성능 증명
- 이미지 인식 대회인 ImageNet Challenge에서 압도적 성능으로 우승(2012년)

컴퓨팅 파워 발전



- CPU는 고성능화와 함께 저가격화

데이터 폭증



- 2020년 데이터의 크기는 40ZB
(40조 GB) 크기

뉴런

3. 인공신경망



인간의 뉴런(neuron)

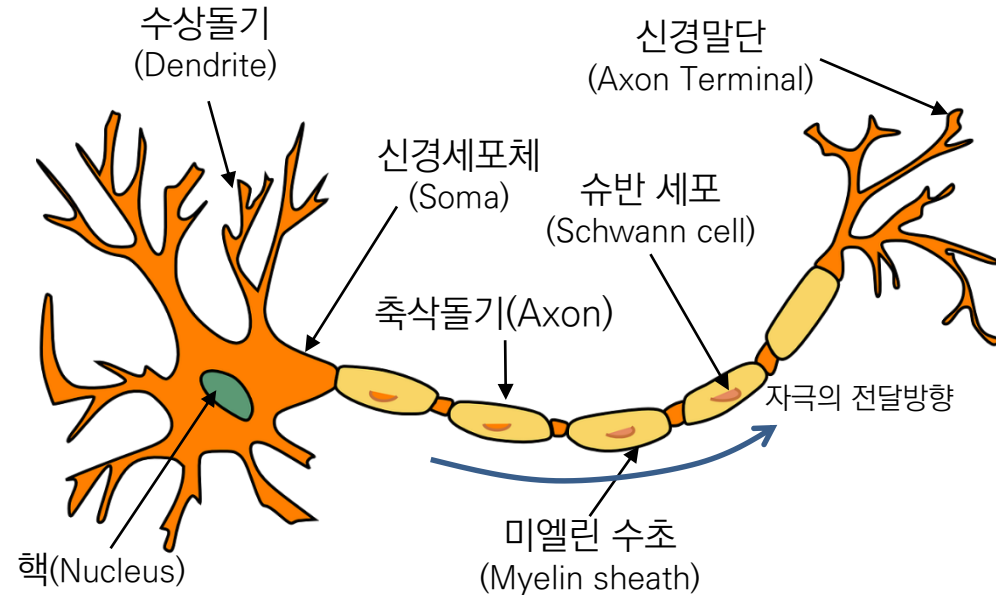
- ▶ 시냅스(synapse)를 통해 뉴런간 신호를 전달
- ▶ 각 뉴런은 수상돌기(dendrite)를 통해 입력 신호를 받음
- ▶ 입력 신호가 특정크기(threshold) 이상인 경우에만 활성화 되어 축삭돌기(axon)을 통해 다음 뉴런으로 전달



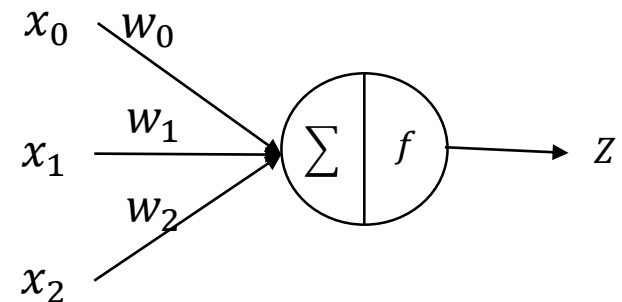
인공 뉴런(노드; node)

- ▶ 각 노드는 가중치가 있는 입력 신호를 받음
- ▶ 입력신호는 모두 더한 후, 활성화 함수(activation function)를 적용함
- ▶ 활성화 함수의 값이 특정 값 이상인 경우에만 다음 노드의 입력값으로 전달

인간의 뉴런 구조



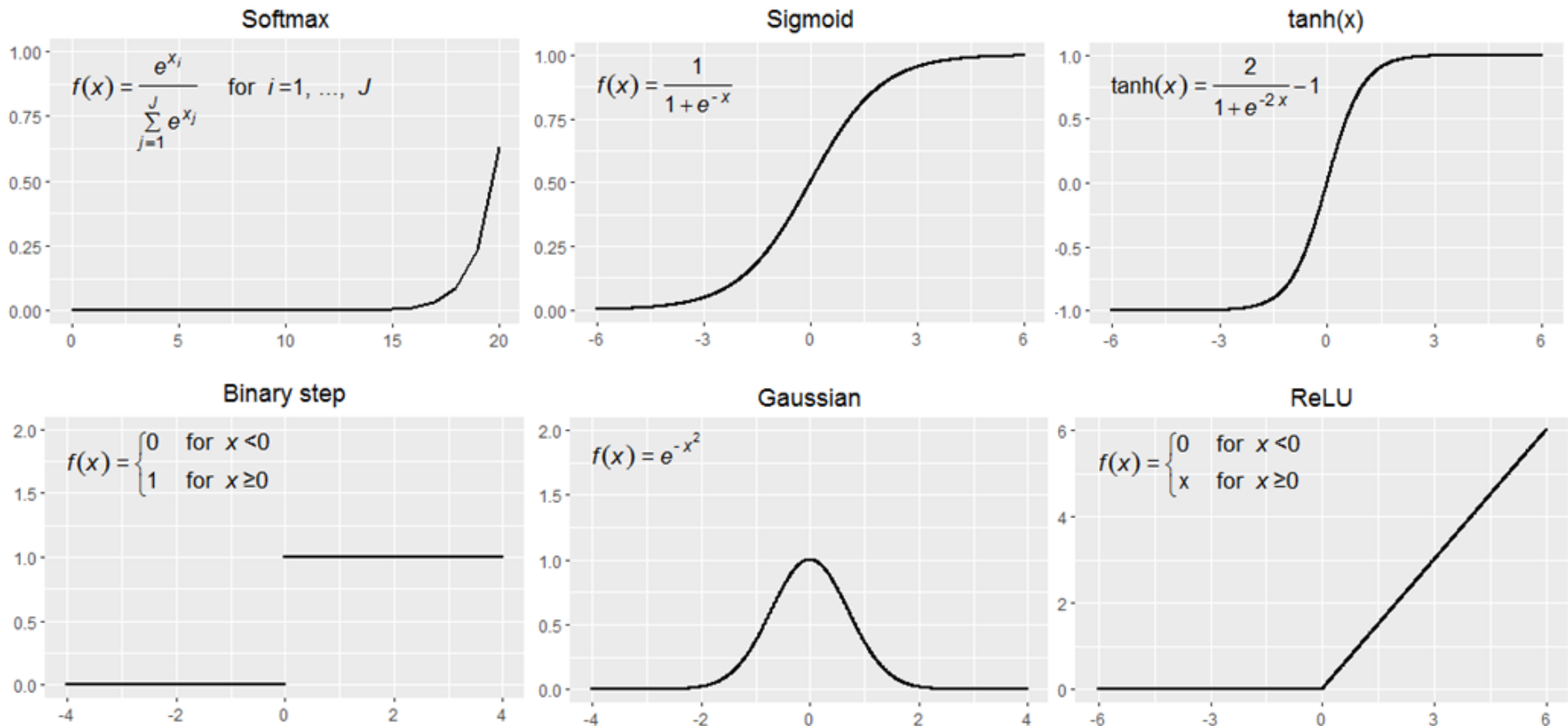
인공 뉴런의 구조



활성화 함수(activation function)

3. 인공신경망

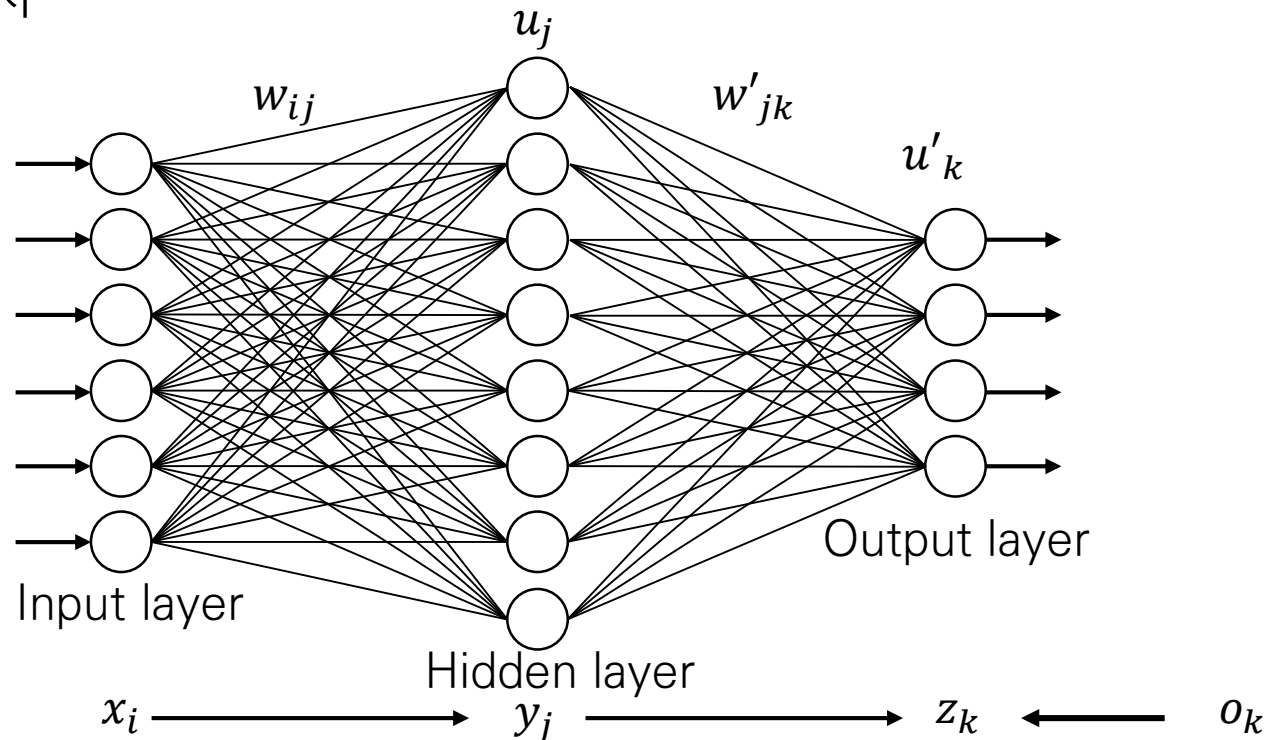
- 생물학적 뉴런(neuron)에서 입력 신호가 일정 크기 이상일 때만 신호를 전달하는 메커니즘을 모방한 함수



인공신경망

3. 인공신경망

- 인공신경망은 입력층(Input layer), 은닉층(Hidden layer), 출력층으로 구성
- 각 층의 뉴런들을 퍼셉트론(Perceptron)
 - 퍼셉트론(Perceptron)은 인공 뉴런의 한 종류입니다.
- 입력층의 뉴런의 수는 입력 데이터의 수이며, 출력층은 분류 문제를 해결할 경우에는 분류의 수와 일치



인공신경망 구성요소

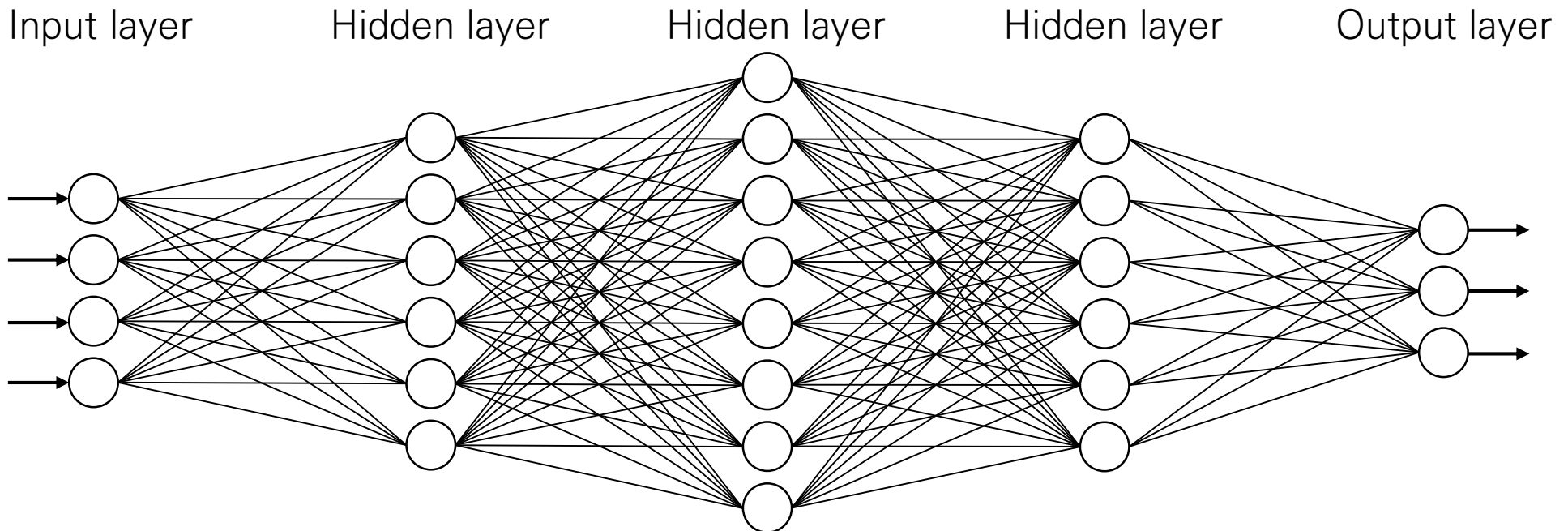
3. 인공신경망

- 입력층(input layer)
 - 입력 값으로 구성된 레이어
 - 학습 데이터셋의 “입력 변수의 개수” 만큼의 노드로 구성
 - 바이어스를 사용한다면 입력 변수의 수+1개 노드로 구성
- 출력층(output layer)
 - 모델의 출력값을 만들어 내는 레이어
 - 예) IRIS 품종 분류 문제
 - 멀티 클래스 분류 문제의 경우 소프트맥스(softmax) 함수를 출력함수로 사용
- 은닉층(hidden layer)
 - 입력층과 출력층 사이의 레이어
 - 뉴런(neuron)과 시냅스(synapse)로 구성된 인간의 두뇌를 모방하는 레이어
 - 은닉층으로 들어오는 입력값의 합을 계산한 후 활성화 함수를 적용
 - 활성화 함수 출력이 임계치를 넘지 않을 경우 다음 노드로 0을 전달(신호를 전달하지 않음)
 - 은닉층의 개수와 은닉 노드 개수
 - 너무 적으면 입력 데이터를 제대로 표현하지 못해 모델을 제대로 학습하지 못함
 - 너무 많으면 과적합(overfitting)이 발생하며, 학습 시간도 많이 소모

다층 신경망

3. 인공신경망

- 은닉층이 여러 개로 구성된 인공 신경망
- DNN(Deep Neural Network)은 다층 인공신경망임
- 실제로 구현된 DNN의 경우에는 이렇게 간단하지 않음
- 은닉 계층이 60~90개 정도로 이뤄진 신경망도 많이 있음



Keras

4. 케라스를 이용한 인공지능망 구현

- 사용자가 손쉽게 딥 러닝을 구현할 수 있도록 도와주는 상위 레벨의 인터페이스
- <https://keras.io/>
- Keras is compatible with: Python 2.7–3.9.
- Tensorflow 2.x 부터 Tensorflow안에 Keras가 포함되어 배포됨



Sequential model

4. 케라스를 이용한 인공신경망 구현

- 예

```
from keras.models import Sequential
from keras.layers import Dense
model=Sequential()
model.add(Dense(3, input_dim=4, activation='softmax'))
```

- 장점

- sequential API로 모델을 설계하는 것은 직관적이고 편리함

- 단점

- 단순히 층을 쌓는 것만으로는 모든 인공신경망을 구현할 수 없다. 즉, 복잡한 인공 신경망을 구현할 수 없다.

<https://keras.io/getting-started/sequential-model-guide/>

Optimizer

4. 케라스를 이용한 인공신경망 구현

옵티마이저	클래스(기본 파라미터)	설명
SGD	<code>keras.optimizers.SGD(lr=0.01, momentum=0.0, decay=0.0, nesterov=False)</code>	Stochastic gradient descent optimizer.
RMSprop	<code>keras.optimizers.RMSprop(lr=0.001, rho=0.9, epsilon=None, decay=0.0)</code>	RMSProp optimizer.
Adagrad	<code>keras.optimizers.Adagrad(lr=0.01, epsilon=None, decay=0.0)</code>	Adagrad optimizer.
Adadelta	<code>keras.optimizers.Adadelta(lr=1.0, rho=0.95, epsilon=None, decay=0.0)</code>	Adadelta optimizer.
Adam	<code>keras.optimizers.Adam(lr=0.001, beta_1=0.9, beta_2=0.999, epsilon=None, decay=0.0, amsgrad=False)</code>	Adam optimizer.
Adamax	<code>keras.optimizers.Adamax(lr=0.002, beta_1=0.9, beta_2=0.999, epsilon=None, decay=0.0)</code>	Adamax optimizer from Adam paper's Section 7.
Nadam	<code>keras.optimizers.Nadam(lr=0.002, beta_1=0.9, beta_2=0.999, epsilon=None, schedule_decay=0.004)</code>	Nesterov Adam optimizer.

Activation functions

4. 케라스를 이용한 인공신경망 구현

이름	함수	설명
softmax	<code>keras.activations.softmax(x, axis=-1)</code>	Softmax activation function.
elu	<code>keras.activations.elu(x, alpha=1.0)</code>	Exponential linear unit.
selu	<code>keras.activations.selu(x)</code>	Scaled Exponential Linear Unit (SELU).
softplus	<code>keras.activations.softplus(x)</code>	Softplus activation function.
softsign	<code>keras.activations.softsign(x)</code>	Softsign activation function.
relu	<code>keras.activations.relu(x, alpha=0.0, max_value=None, threshold=0.0)</code>	Rectified Linear Unit.
tanh	<code>keras.activations.tanh(x)</code>	Hyperbolic tangent.
sigmoid	<code>keras.activations.sigmoid(x)</code>	Sigmoid activation function.
hard_sigmoid	<code>keras.activations.hard_sigmoid(x)</code>	Faster to compute than sigmoid activation.
exponential	<code>keras.activations.exponential(x)</code>	Exponential (base e) activation function.
linear	<code>keras.activations.linear(x)</code>	Linear (i.e. identity) activation function.

<https://keras.io/activations/>

Advanced Activation functions

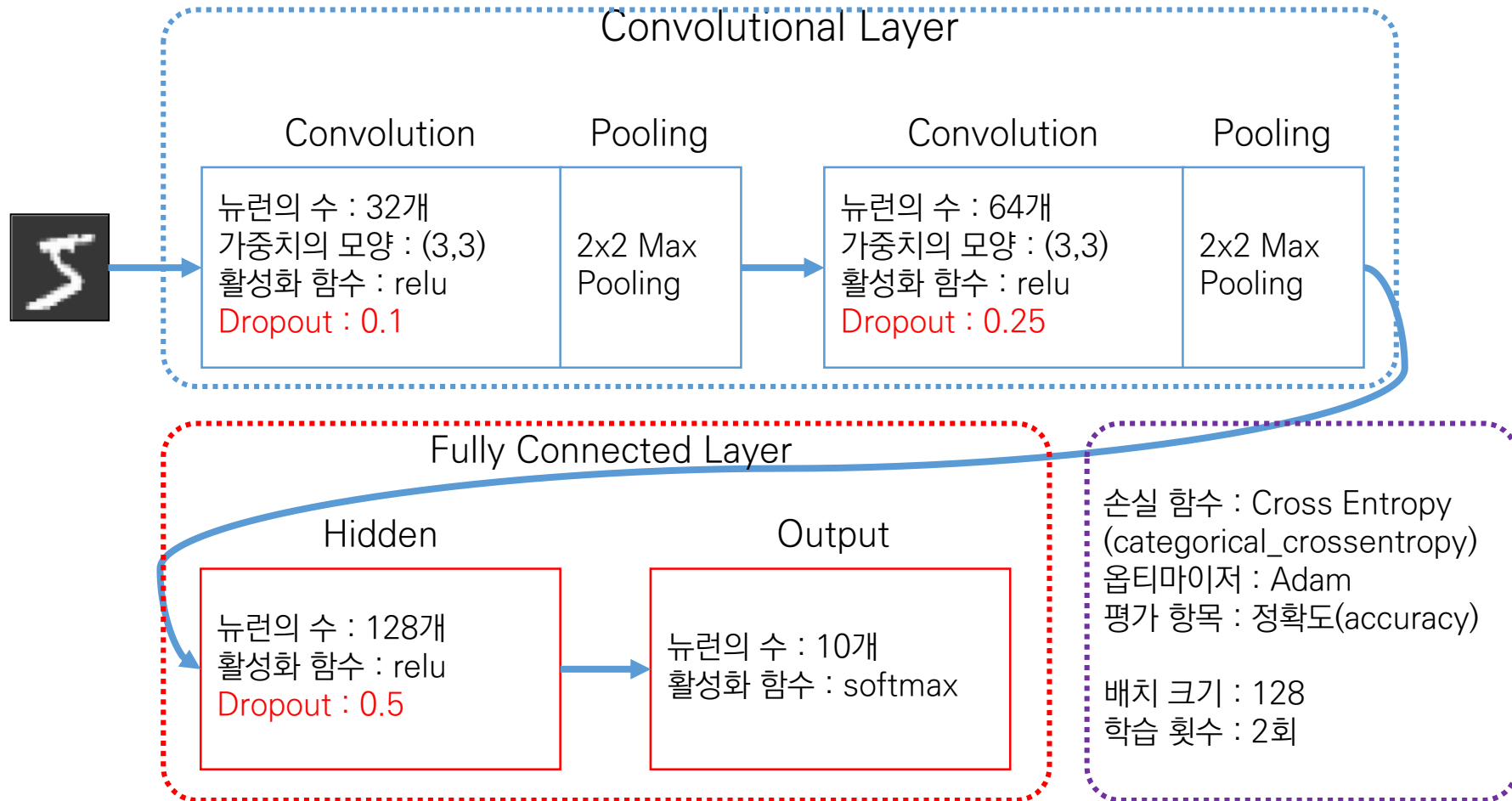
4. 케라스를 이용한 인공신경망 구현

이름	함수	설명
LeakyReLU	keras.layers.LeakyReLU(alpha=0.3) Rectifier Nonlinearities Improve Neural Network Acoustic Models (https://ai.stanford.edu/~amaas/papers/relu_hybrid_icml2013_final.pdf)	Leaky version of a Rectified Linear Unit.
PReLU	keras.layers.PReLU(alpha_initializer='zeros', alpha_regularizer=None, alpha_constraint=None, shared_axes=None) Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification(https://arxiv.org/abs/1502.01852)	Parametric Rectified Linear Unit.
ELU	keras.layers.ELU(alpha=1.0) Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs)(https://arxiv.org/abs/1511.07289v1)	Exponential Linear Unit.
ThresholdedReLU	keras.layers.ThresholdedReLU(theta=1.0) Zero-Bias Autoencoders and the Benefits of Co-Adapting Features (https://arxiv.org/abs/1402.3337)	Thresholded Rectified Linear Unit.
Softmax	keras.layers.Softmax(axis=-1)	Softmax activation function.
ReLU	keras.layers.ReLU(max_value=None, negative_slope=0.0, threshold=0.0)	Rectified Linear Unit activation function.

<https://keras.io/layers/advanced-activations/>

CNN 모델 구현하기

4. 케라스를 이용한 인공신경망 구현



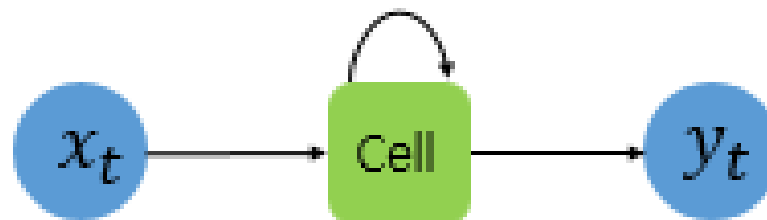


5. RNN

순환 신경망(RNN)

5. RNN

- RNN(Recurrent Neural Network)은 시퀀스(Sequence) 모델임
 - 입력과 출력을 시퀀스로 처리하는 모델
 - 번역기를 생각해보면 입력은 번역하고자 하는 문장. 즉, 시퀀스입니다. 출력에 해당되는 번역된 문장 또한 시퀀스입니다.
 - 시퀀스 모델들을 처리하기 위해 고안된 모델들을 시퀀스 모델이라고 함
 - RNN은 시퀀스 모델의 가장 대표적이고 기본적인 모델
- RNN은 은닉층에서 활성화 함수를 통해 나온 결과값을 출력층 방향으로 보내면서, 다시 은닉층의 다음 계산의 입력으로 보내는 특징을 갖고 있음

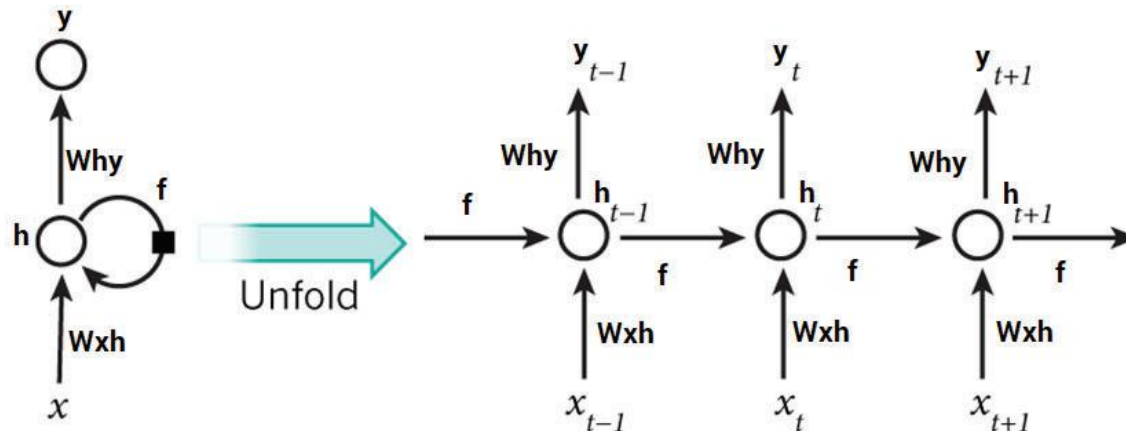


편향 b 도 은닉층과 출력층의 입력으로 존재하지만 앞으로의 신경망의 그림에서 편향 b 는 생략함

은닉 상태

5. RNN

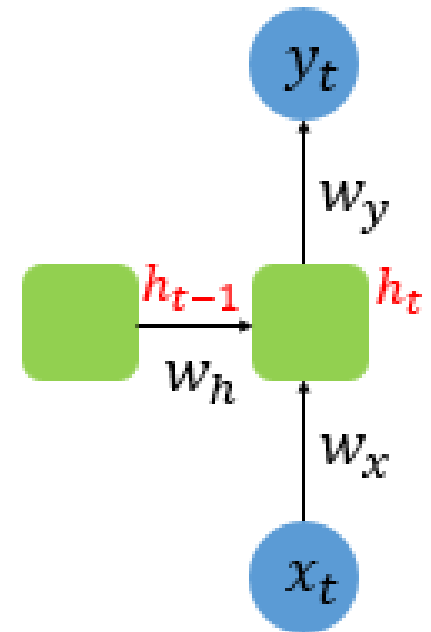
- 은닉층의 메모리 셀은 각각의 시점(time-step)에서 바로 이전 시점에서의 은닉층의 메모리 셀에서 나온 값들을 계속해서 자신의 입력으로 보내는 재귀적 활동을 하고 있음
 - 현재 시점을 t 로 표현하고, 이전 시점을 $t-1$, 다음 시점을 $t+1$ 와 같은 형식으로 표현함
 - 이는 현재 시점 t 에서의 메모리 셀이 갖고있는 값은 과거의 메모리 셀들의 값에 영향을 받은 것임을 의미함
- 메모리 셀이 다음 시점 $t+1$ 에 다시 자신에게 보내는 이 값을 은닉 상태(hidden state)라고 함
 - 다시 말해 현재 시점 t 의 메모리 셀은 이전 시점 $t-1$ 에서의 메모리 셀이 보낸 은닉 상태값을 다시 계산을 위한 입력값으로 사용



RNN의 은닉층, 출력층에 대한 수식

5. RNN

- 현재 시점 t 에서의 은닉 상태값을 h_t 라고 정의
- 은닉층의 메모리 셀은 h_t 를 계산하기 위해서 총 두 개의 가중치를 갖게 됨
 - 입력층에서 입력값을 위한 가중치 W_x 이고,
 - 이전 시점 $t-1$ 의 은닉 상태값인 h_{t-1} 을 위한 가중치 W_h
- 식
 - 은닉층 : $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$
 - 출력층 : $y_t = f(W_y h_t + b)$
단, f 는 비선형 활성화 함수 중 하나.



RNN의 은닉층, 출력층에 대한 수식

5. RNN

- 배치 크기가 1이고, d 와 D_h 두 값 모두를 4로 가정하였을 때, RNN의 은닉층 연산을 그림으로 표현

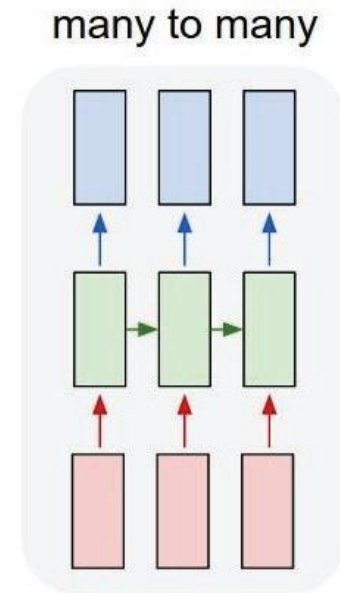
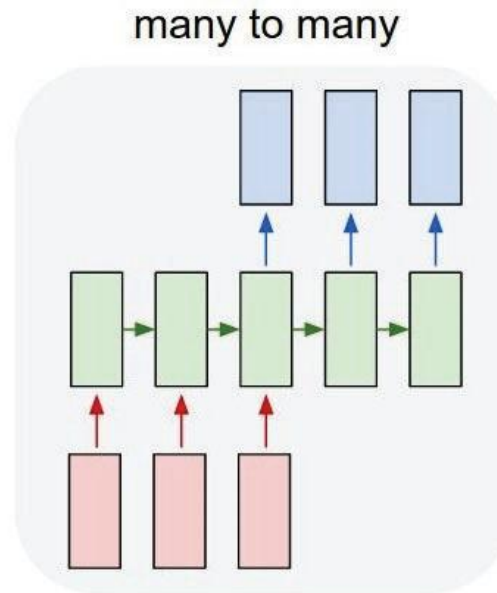
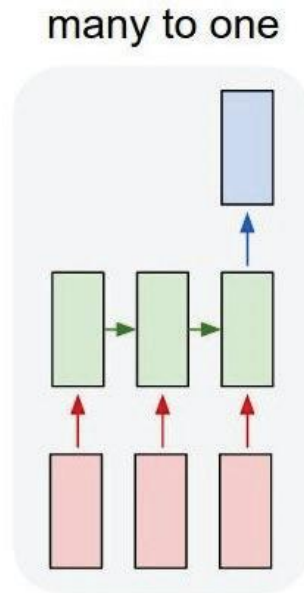
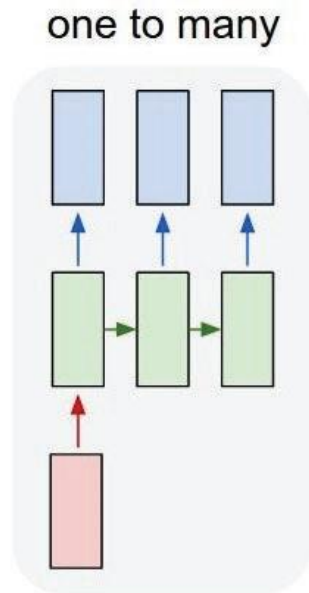
$$\tanh \left(\begin{matrix} W_h \\ D_h \times D_h \end{matrix} \times \begin{matrix} h_{t-1} \\ D_h \times 1 \end{matrix} + \begin{matrix} W_x \\ D_h \times d \end{matrix} \times \begin{matrix} x_t \\ d \times 1 \end{matrix} + \begin{matrix} b \\ D_h \times 1 \end{matrix} \right) = \begin{matrix} h_t \\ D_h \times 1 \end{matrix}$$

RNN의 종류

5. RNN

● 다루는 문제에 따라 다름

- One-to-many : 한 음으로 다음 여러 개의 음을 작곡
- Many-to-one : 여러 입력을 통해 하나의 출력값 생성
- Many-to-many : 번역기와 챗봇에 주로 사용



문맥을 예측해서 다음 단어 예측해보기

5. RNN

```
1 text="""경마장에 있는 말이 뛰고 있다\n
2 그의 말이 법이다\n
3 가는 말이 고와야 오는 말이 곱다\n"""
```

```
1 from keras_preprocessing.text import Tokenizer
2 t = Tokenizer()
3 t.fit_on_texts([text])
4 encoded = t.texts_to_sequences([text])[0]
```

토큰화와 정수
인코딩

```
1 vocab_size = len(t.word_index) + 1
2 # 케라스 토큰라이저의 정수 인코딩은 인덱스가 1부터 시작하지만,
3 # 케라스 원-핫 인코딩에서 배열의 인덱스가 0부터 시작하기 때문에
4 # 배열의 크기를 실제 단어 집합의 크기보다 +1로 생성해야하므로 미리 +1 선언
5 print('단어 집합의 크기 : %d' % vocab_size)
```

단어 집합의 크기 : 12

```
1 print(t.word_index)
```

```
{'말이': 1, '경마장에': 2, '있는': 3, '뛰고': 4, '있다': 5, '그의': 6, '법이다': 7,
'가는': 8, '고와야': 9, '오는': 10, '곱다': 11}
```

문맥을 예측해서 다음 단어 예측해보기

5. RNN

```
1 print(t.word_index)
```

```
{'말이': 1, '경마장에': 2, '있는': 3, '뛰고': 4, '있다': 5, '그의': 6, '법이다': 7,
'가는': 8, '고와야': 9, '오는': 10, '곰다': 11}
```

```
1 sequences = list()
2 for line in text.split('\n'): # \n을 기준으로 문장 토큰화
3     encoded = t.texts_to_sequences([line])[0]
4     for i in range(1, len(encoded)):
5         sequence = encoded[:i+1]
6         sequences.append(sequence)
7
8 print('훈련 데이터의 개수: %d' % len(sequences))
```

훈련 데이터의 개수: 11

```
1 print(sequences)
```

```
[[2, 3], [2, 3, 1], [2, 3, 1, 4], [2, 3, 1, 4, 5], [6, 1], [6, 1, 7], [8, 1], [8, 1,
9], [8, 1, 9, 10], [8, 1, 9, 10, 1], [8, 1, 9, 10, 1, 11]]
```

```
1 print(max(len(l) for l in sequences))
```

[2, 3]은 [경마장에, 있는]에
해당되며 [2, 3, 1]은 [경마장에,
있는, 말이]에 해당됨
모든 훈련 데이터에 대해서 맨
우측에 있는 단어에 대해서만 y로
분리하면 X와 y의 쌍(pair)이 됨

문맥을 예측해서 다음 단어 예측해보기

5. RNN

```
1 from keras.preprocessing.sequence import pad_sequences
2 sequences = pad_sequences(sequences, maxlen=6, padding='pre')
```

Using TensorFlow backend.

```
1 print(sequences)
```

```
[[ 0  0  0  0  2  3]
 [ 0  0  0  2  3  1]
 [ 0  0  2  3  1  4]
 [ 0  2  3  1  4  5]
 [ 0  0  0  0  6  1]
 [ 0  0  0  6  1  7]
 [ 0  0  0  0  8  1]
 [ 0  0  0  8  1  9]
 [ 0  0  8  1  9 10]
 [ 0  8  1  9 10  1]
 [ 8  1  9 10  1 11]]
```

- pad_sequences()는 모든 데이터에 대해서 0을 추가하여 길이를 맞춰줌
- maxlen의 값으로 6을 주면 모든 데이터의 길이를 6으로 맞춰주며, padding의 인자로 'pre'를 주면 길이가 6보다 짧은 데이터의 앞을 0으로 채움

문맥을 예측해서 다음 단어 예측해보기

5. RNN

```
1 import numpy as np
2 sequences = np.array(sequences)
3 X = sequences[:, :-1]
4 y = sequences[:, -1]
5 # 리스트의 마지막 열을 제외하고 저장한 것은 X
6 # 리스트의 마지막 열만 저장한 것은 y
```

```
1 print(X)
```

```
[[ 0  0  0  0  2]
 [ 0  0  0  2  3]
 [ 0  0  2  3  1]
 [ 0  2  3  1  4]
 [ 0  0  0  0  6]
 [ 0  0  0  6  1]
 [ 0  0  0  0  8]
 [ 0  0  0  8  1]
 [ 0  0  8  1  9]
 [ 0  8  1  9 10]
 [ 8  1  9 10 11]]
```

X와 y의 분리

문맥을 예측해서 다음 단어 예측해보기

5. RNN

```
1 print(y)
```

```
[ 3  1  4  5  1  7  1  9 10  1 11]
```

```
1 from keras.utils import to_categorical
2 y = to_categorical(y, num_classes=vocab_size)
```

```
1 print(y)
```

```
[[0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0.]
 [0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]]
```

훈련 데이터를 훈련
시키기 전에 y에 대해서
원-핫 인코딩을 수행

문맥을 예측해서 다음 단어 예측해보기

5. RNN

```
1 from keras.layers import Embedding, Dense, SimpleRNN
2 from keras.models import Sequential
3
4 model = Sequential()
5 # y를 제거하였으므로 이제 x의 길이는 5
6 model.add(Embedding(vocab_size, 10, input_length=5))
7 model.add(SimpleRNN(32))
8 model.add(Dense(vocab_size, activation='softmax'))
9 model.compile(loss='categorical_crossentropy', optimizer='adam',
10               metrics=['accuracy'])
11 model.fit(X, y, epochs=200, verbose=2)
```

Epoch 180/200

- 0s - loss: 0.1980 - acc: 1.0000

Epoch 181/200

- 0s - loss: 0.1948 - acc: 1.0000

Epoch 182/200

- 0s - loss: 0.1917 - acc: 1.0000

Epoch 183/200

- 0s - loss: 0.1886 - acc: 1.0000

RNN 모델에 데이터를
훈련시킴

중간부터 정확도가
100%가 나옴

문맥을 예측해서 다음 단어 예측해보기

5. RNN

```

1 def sentence_generation(model, t, current_word, n): # 모델, 토큰라이저, 현재 단어, 반복할 횟수
2     init_word = current_word # 처음 들어온 단어도 마지막에 같이 출력하기 위해 저장
3     sentence = ''
4     for _ in range(n): # n번 반복
5         encoded = t.texts_to_sequences([current_word])[0] # 현재 단어에 대한 정수 인코딩
6         encoded = pad_sequences([encoded], maxlen=5, padding='pre') # 데이터에 대한 패딩
7         result = model.predict_classes(encoded, verbose=0)
8         # 입력한 X(현재 단어)에 대해서 Y를 예측하고 Y(예측한 단어)를 result에 저장.
9         for word, index in t.word_index.items():
10             if index == result: # 만약 예측한 단어와 인덱스와 동일한 단어가 있다면
11                 break # 해당 단어가 예측 단어이므로 break
12             current_word = current_word + ' ' + word # 현재 단어 + ' ' + 예측 단어를 현재 단어로 변경
13             sentence = sentence + ' ' + word # 예측 단어를 문장에 저장
14         # for문이므로 이 행동을 다시 반복
15     sentence = init_word + sentence
16     return sentence

```

```

1 print(sentence_generation(model, t, '경마장에', 4))
2 # '경마장에' 라는 단어 뒤에는 총 4개의 단어가 있으므로 4번 예측

```

경마장에 있는 말이 뛰고 있다

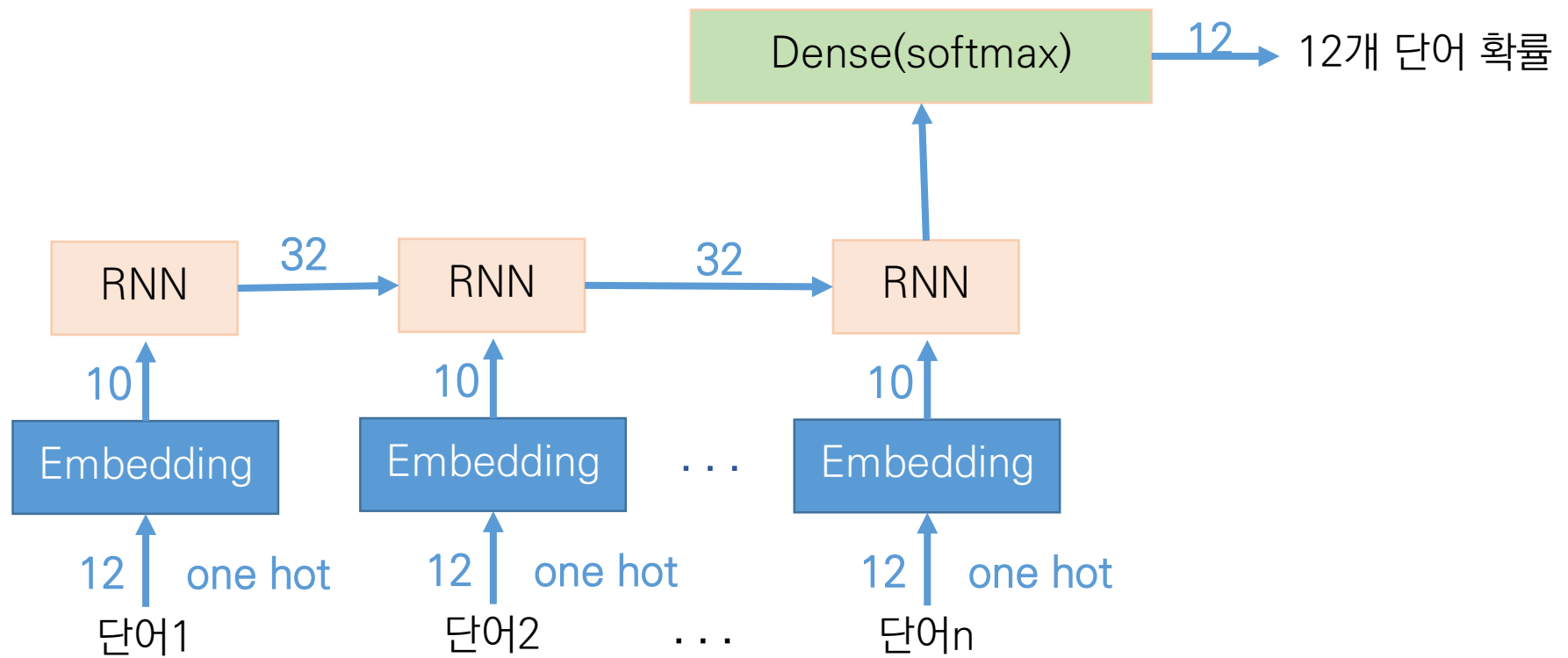
모델이 정확하게 예측하고
있는지 문장을 생성하는 함수를
만들어서 실제로 출력

```

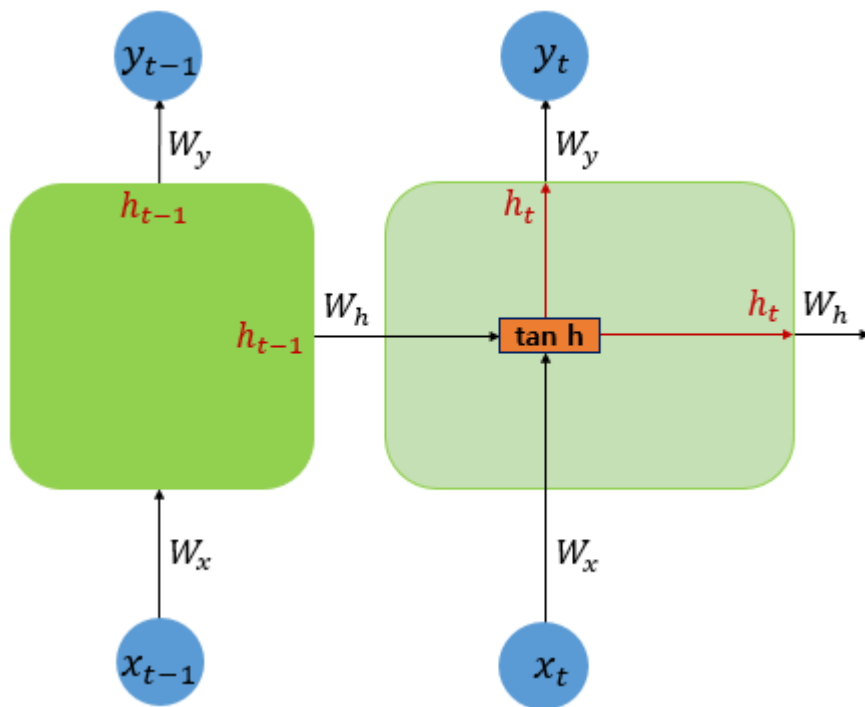
1 print(sentence_generation(model, t, '그의', 2)) # 2번 예측

```

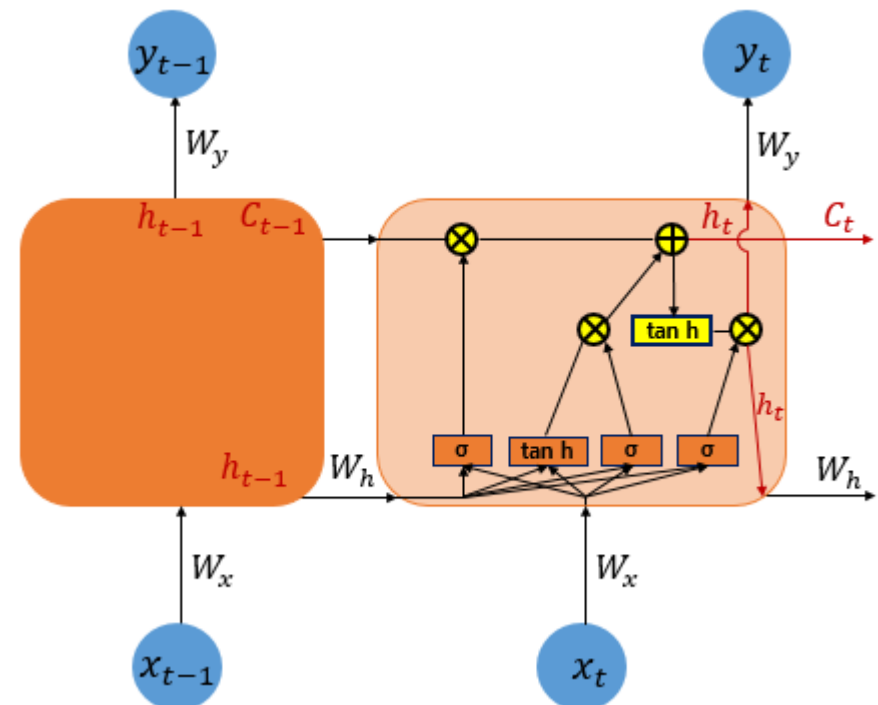
그의 말이 법이다



Vanilla RNN



LSTM(Long Short-Term Memory)



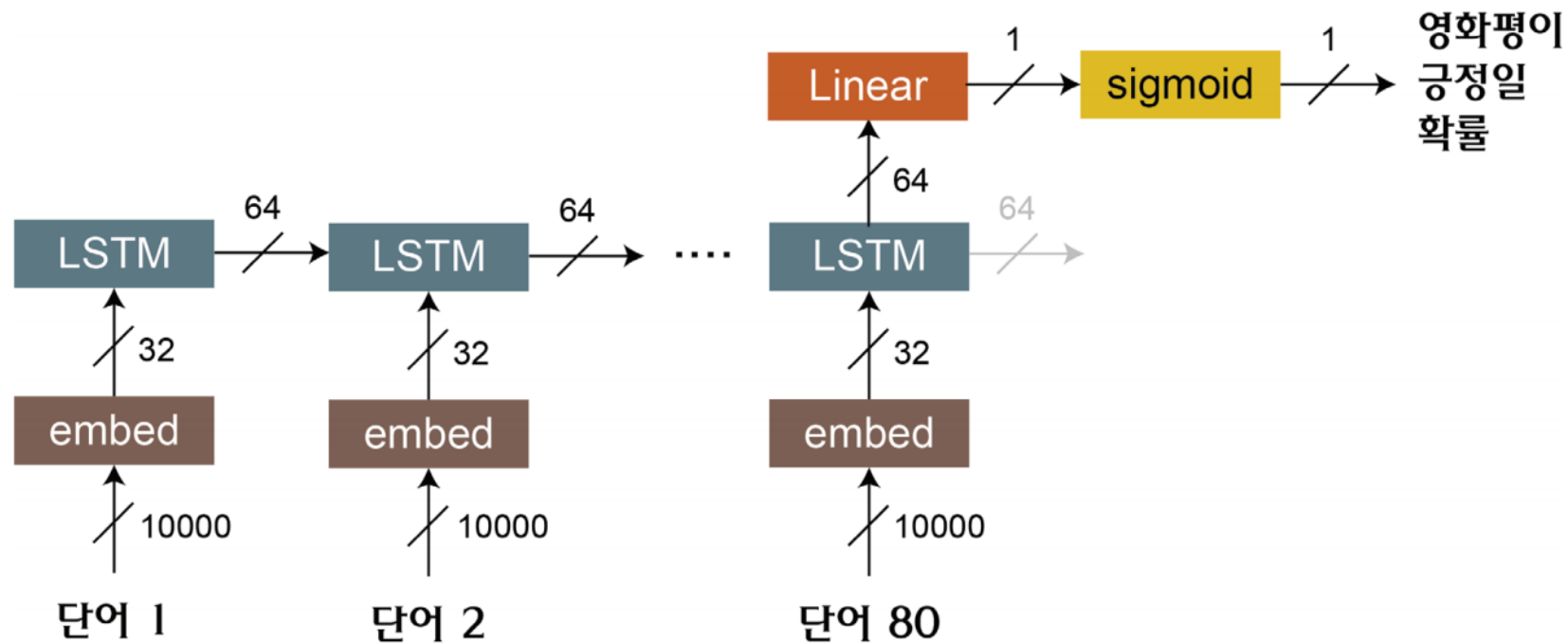
LSTM

6. LSTM

- 기존 코드와 크게 다른점이 없고 모델을 만드는 클래스가 바뀌었음

```
1 from keras.layers import Embedding, Dense, LSTM
2 from keras.models import Sequential
3
4 model = Sequential()
5 model.add(Embedding(vocab_size, 10, input_length=max_len-1))
6 # y를 제거하였으므로 이제 X의 길이는 기존 데이터의 길이 - 1
7 model.add(LSTM(128))
8 model.add(Dense(vocab_size, activation='softmax'))
9 model.compile(loss='categorical_crossentropy', optimizer='adam',
10               metrics=['accuracy'])
11 model.fit(X, y, epochs=200, verbose=2)
```

- Text 영화평 긍정/부정 분류
 - 입력: 80개 단어
 - 출력: [0, 1] 확률 (1=긍정)



- Sparse vector를 dense vector로 전환
 - 1-hot encoding (sparse): 너무 비 효율적
 - Word embedding (sense): 메모리 사용과 계산 속도를 획기적으로 단축

단어	key	one-hot encoding	임베딩
hope	0	[1, 0, 0, 0, 0, 0, 0]	[1.2, 3.1]
to	1	[0, 1, 0, 0, 0, 0, 0]	[0.1, 4.2]
see	2	[0, 0, 1, 0, 0, 0, 0]	[1.0, 3.1]
you	3	[0, 0, 0, 1, 0, 0, 0]	[0.3, 2.1]
soon	4	[0, 0, 0, 0, 1, 0, 0]	[2.2, 1.4]
nice	5	[0, 0, 0, 0, 0, 1, 0]	[0.7, 1.7]
again	6	[0, 0, 0, 0, 0, 0, 1]	[4.1, 2.0]

- 단어 임베딩 기술의 하나 (2013)

- 단어를 벡터로 변환하는 기술
- 입력 데이터의 차원 수가 줄어 듦
- 비슷한 의미의 단어들을 벡터 공간 안에서 가까이 모아 주는 역할

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 \end{bmatrix} \times \begin{bmatrix} 8 & 2 & 1 & 9 \\ 6 & 5 & 4 & 0 \\ 7 & 1 & 6 & 2 \\ 1 & 3 & 5 & 8 \\ 0 & 4 & 9 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 3 & 5 & 8 \end{bmatrix}$$

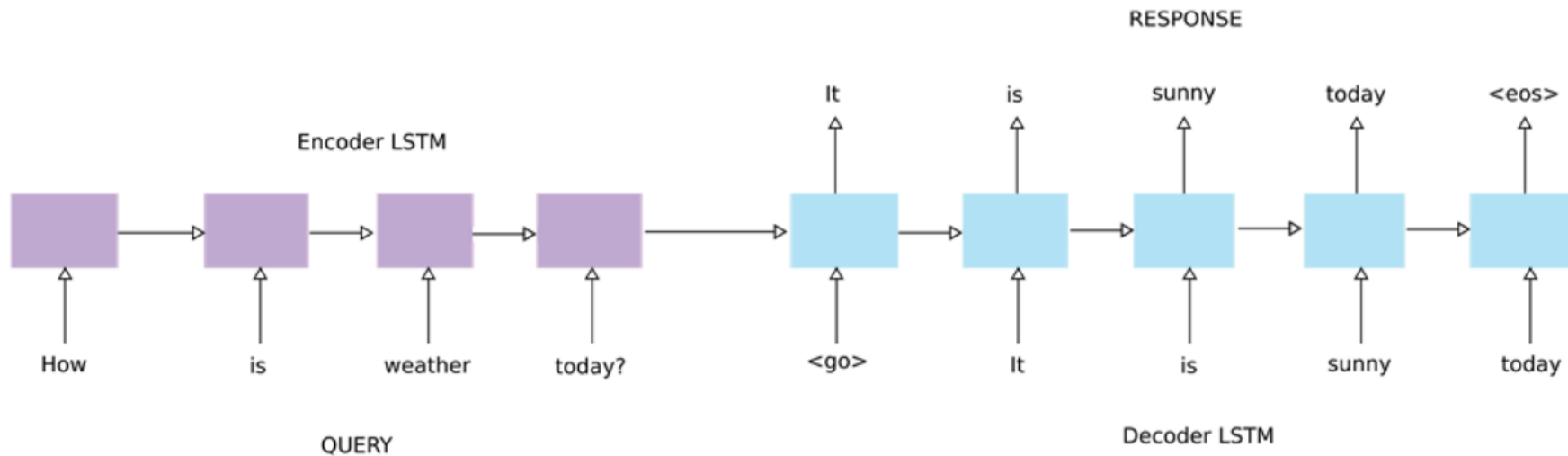
One-hot vector

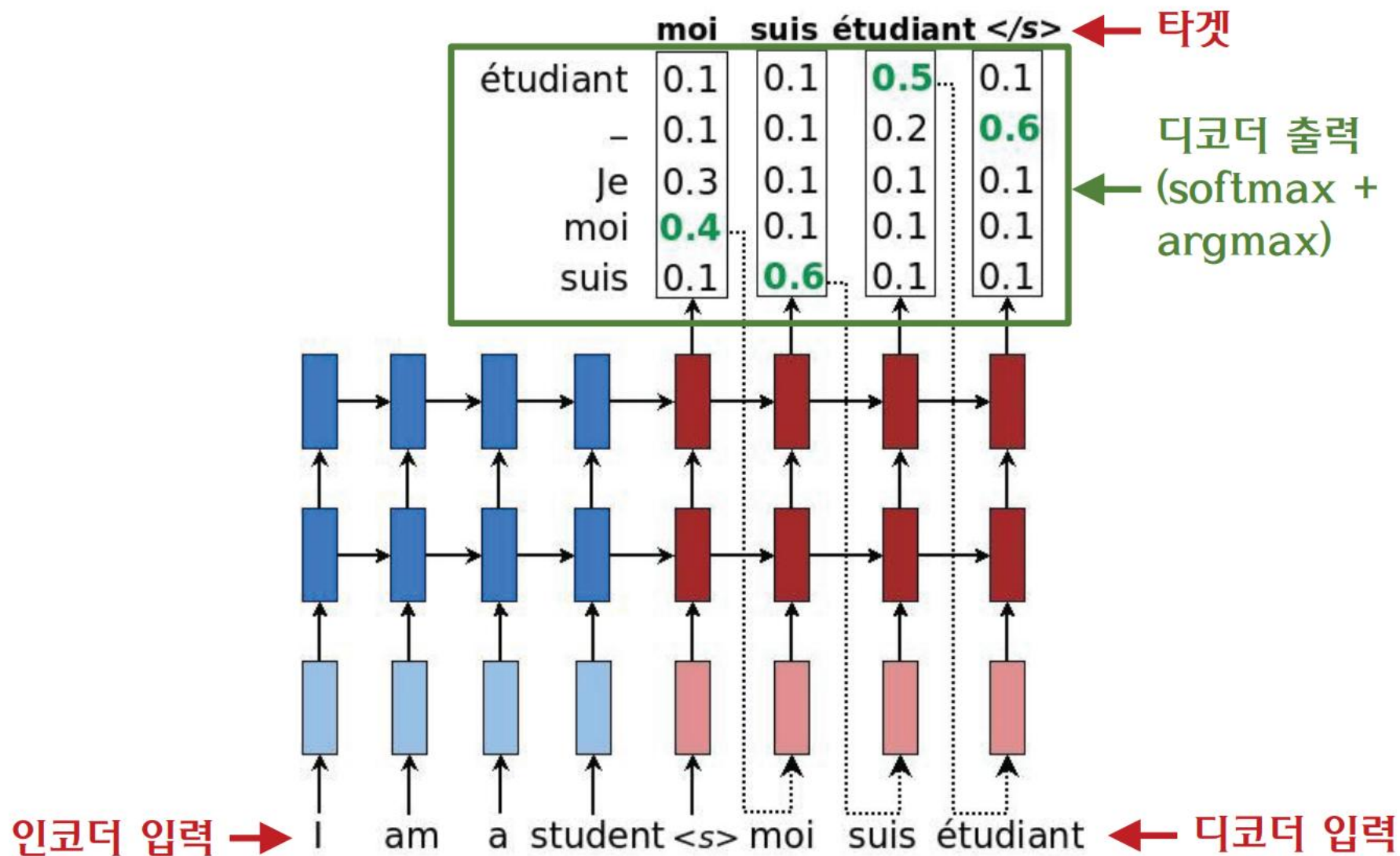
Embedding Weight Matrix

Hidden layer output

이 행렬이 학습됨

- Google Neural Machine Translation (GNMT)
 - RNN 기반 sequence-to-sequence 방식
 - 인코더/디코더 연결 구조
 - 인코더: 입력 문장
 - 디코더: 출력 문장
 - 응용: 번역, 챗봇





이번에 풀 문제

- 영어 단어를 한글로 번역
 - 입력: 4글자 영어 단어
 - 출력: 2글자 한글 단어
- 예
 - word: 단어, wood: 나무, game: 놀이, girl: 소녀, kiss: 키스, love: 사랑, 등등
- 이런 시도는?
 - love, olve, lvoe, love, eovl



데이터 처리 과정 전 사전만들기

1. 알파벳 만들기

- 'SEPabcdefghijklmnopqrstuvwxyz단어나무놀이소녀키스사랑'

2. 각 글자 떼어 놓기

- ['S', 'E', 'P', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z', '단', '어', '나', '무', '놀', '이', '소', '녀', '키', '스', '사', '랑']

3. 글자를 순서로 전환하는 사전 만들기

- {'S': 0, 'E': 1, 'P': 2, 'a': 3, 'b': 4, 'c': 5, 'd': 6, 'e': 7, 'f': 8, 'g': 9, 'h': 10, 'i': 11, 'j': 12, 'k': 13, 'l': 14, 'm': 15, 'n': 16, 'o': 17, 'p': 18, 'q': 19, 'r': 20, 's': 21, 't': 22, 'u': 23, 'v': 24, 'w': 25, 'x': 26, 'y': 27, 'z': 28, '단': 29, '어': 30, '나': 31, '무': 32, '놀': 33, '이': 34, '소': 35, '녀': 36, '키': 37, '스': 38, '사': 39, '랑': 40}

데이터 처리 과정 : 단어 전환1

- 예: ['wood', '나무']
 1. 영어 단어 숫자로 전환하기 (인코더 입력)
 - wood: [25, 17, 17, 6]
 2. 한글 단어 숫자로 전환하기 (디코더 입력)
 - 특수 문자 S (= 0) 앞에 추가
 - 나무: [0, 31, 32]
 3. 타겟 만들기 (디코더 출력)
 - 특수 문자 E (= 1) 뒤에 추가
 - 나무: [31, 32, 1]

데이터 처리 과정 : 단어 전환2

4. 인코더 입력 1-hot encoding 하기

– [25, 17, 17, 6]

```
[[ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.  0.  0.  1.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  1.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  1.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  1.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.]
```

5. 디코더 입력 1-hot encoding 하기

– [0, 31, 32]

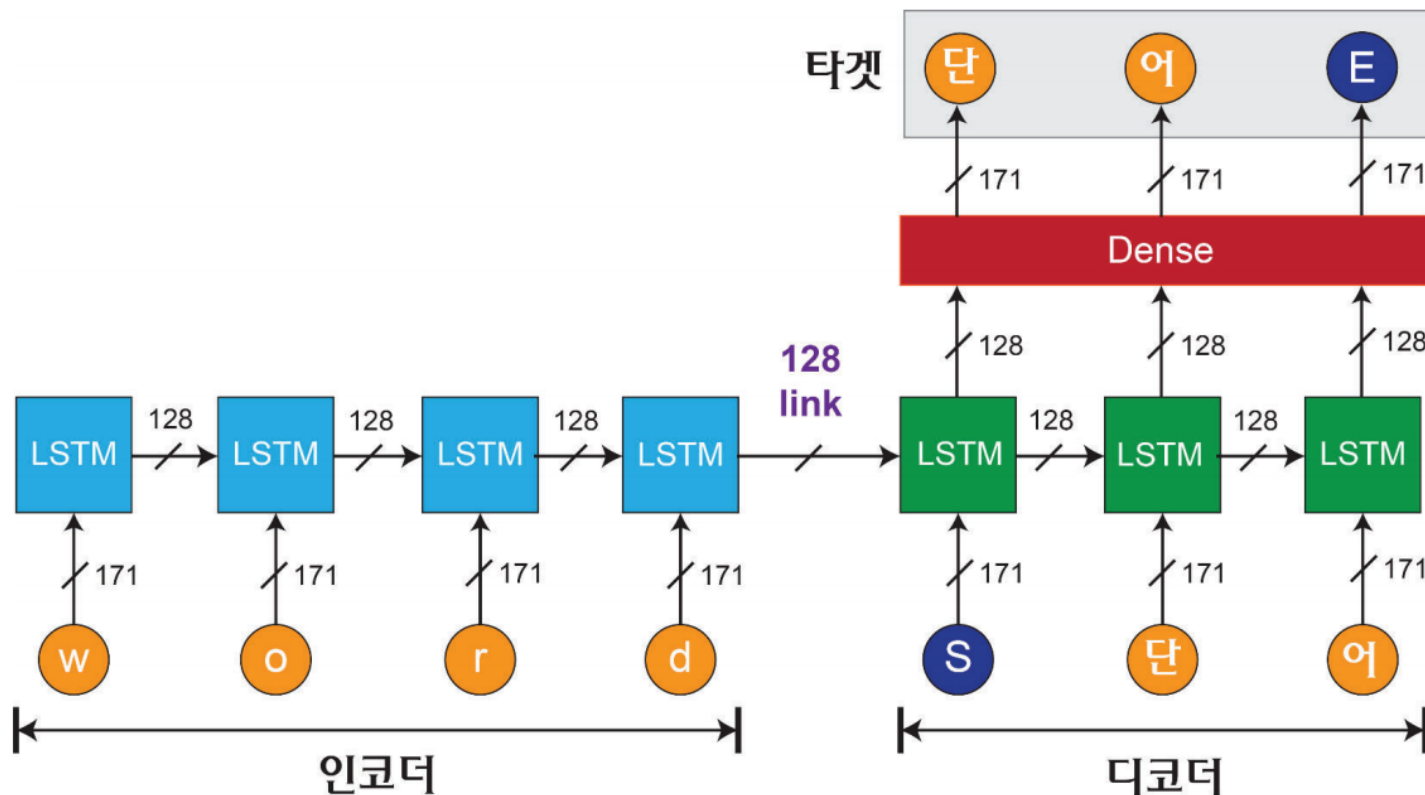
```
[[ 1.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.]
```

```
[ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  1.  0.  0.  0.
  0.  0.  0.  0.  0.  0.]
```

```
[ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  1.  0.  0.
  0.  0.  0.  0.  0.  0.]
```

- Seq2seq 구조

- 입력: 영어 단어 (4 글자) + 한글 단어 (1+2 글자)
- 출력: 한글 단어 (2+1 글자), softmax 활성화



LSTM 2 개 옵션 설정

